

AMBA AHB Design Specifications

September 2026

REVISION

Version	DoU	Content
v1.0	5/26/2026	First Release of the AHB Specification

REVISION	1
1. INTRODUCTION	3
1.1. About AHB Protocols	4
1.2. AHB System Architecture	4
1.3. AHB Signals Interface	5
2. DESIGN SPECIFICATIONS	7
2.1. Design features.....	7
2.1.1. Single transfer (without wait state)	7
2.1.2. Single transfer (with wait state)	7
2.1.3. Transfer types.....	7
2.1.4. Transfer size.....	8
2.1.5. Burst operation.....	9
2.1.6. Burst termination after BUSY transfer	10
2.2. AHB Master	10
2.3. AHB Slave	16
2.4. Address Decoder	21
3. AHB SYSTEM OPERATION.....	22
3.1. Single transfer (Write & Read)	23
3.2. Address Decode transfer	25
3.3. Master waiting transfer	31
3.4. Slave waiting transfer	31
3.5. INCR burst transfer.....	34
3.6. INCR4/INCR8/INCR16 burst transfer	36
3.7. WRAP4/WRAP8/WRAP16 transfer	42

1. INTRODUCTION

1.1. About AHB Protocols

AMBA AHB is a bus interface suitable for high-performance synthesizable designs. It defines the interface between components, such as Managers, interconnects, and Subordinates.

AMBA AHB implements the features required for high-performance, high clock frequency systems including:

- Burst transfers.
- Single clock-edge operation.
- Non-tristate implementation.
- Configurable data bus widths.
- Configurable address bus widths.

The most common AHB Subordinates are internal memory devices, external memory interfaces, and high-bandwidth peripherals.

1.2. AHB System Architecture

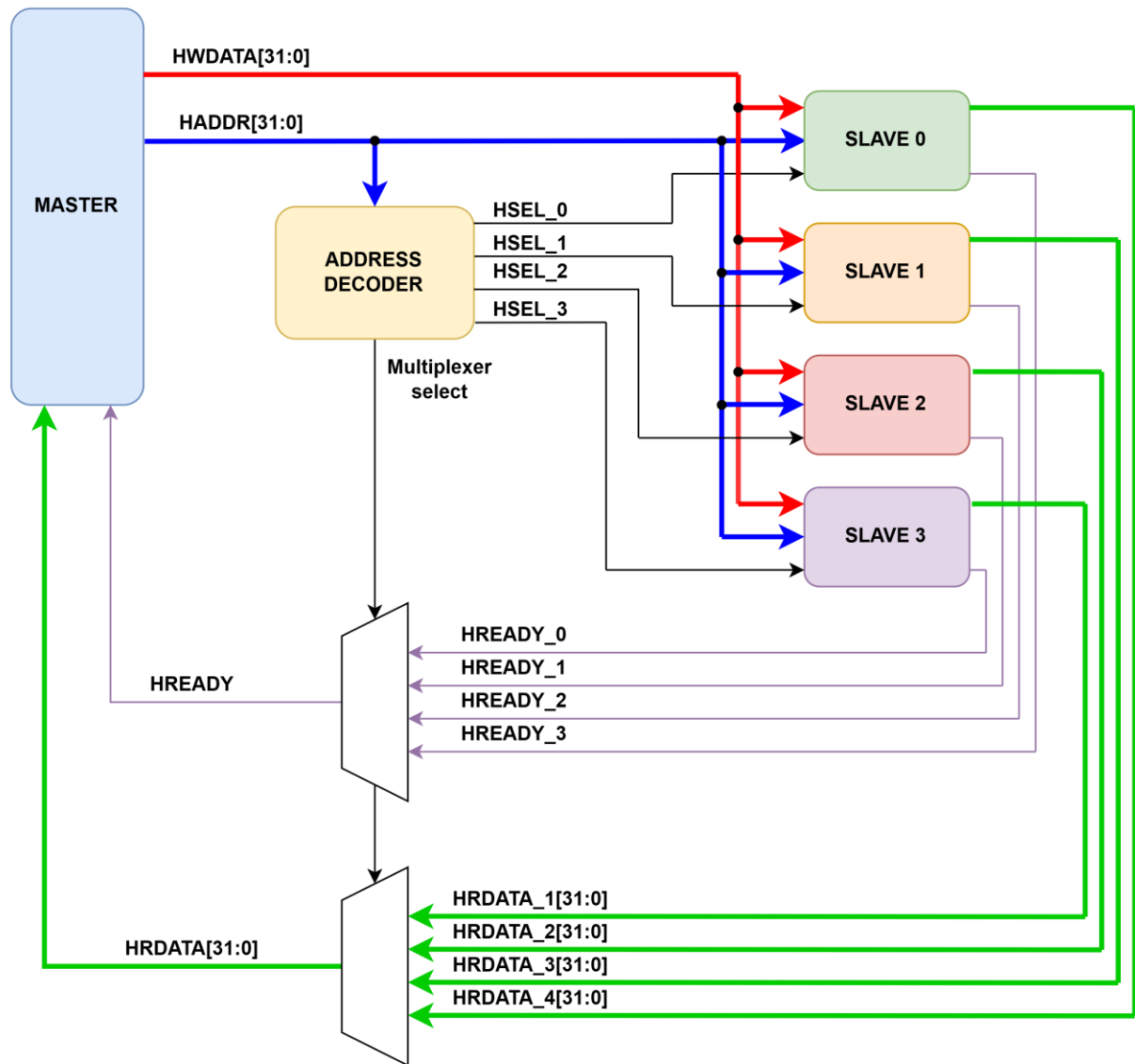


Figure 1.1. AHB block diagram

Figure 1.1. shows a single Manager AHB system design with the AHB Manager and three AHB Subordinates. The bus interconnect logic consists of one address decoder and a Subordinate-to-Manager multiplexor. The decoder monitors the address from the Manager during the address phase so that the appropriate Subordinate is selected during the data phase of a transfer. The multiplexor routes the corresponding Subordinate output data back to the Manager.

AHB also supports multi-Manager designs by using an interconnect component that provides arbitration and routing signals from different Managers to the appropriate Subordinates.

1.3. AHB Signals Interface

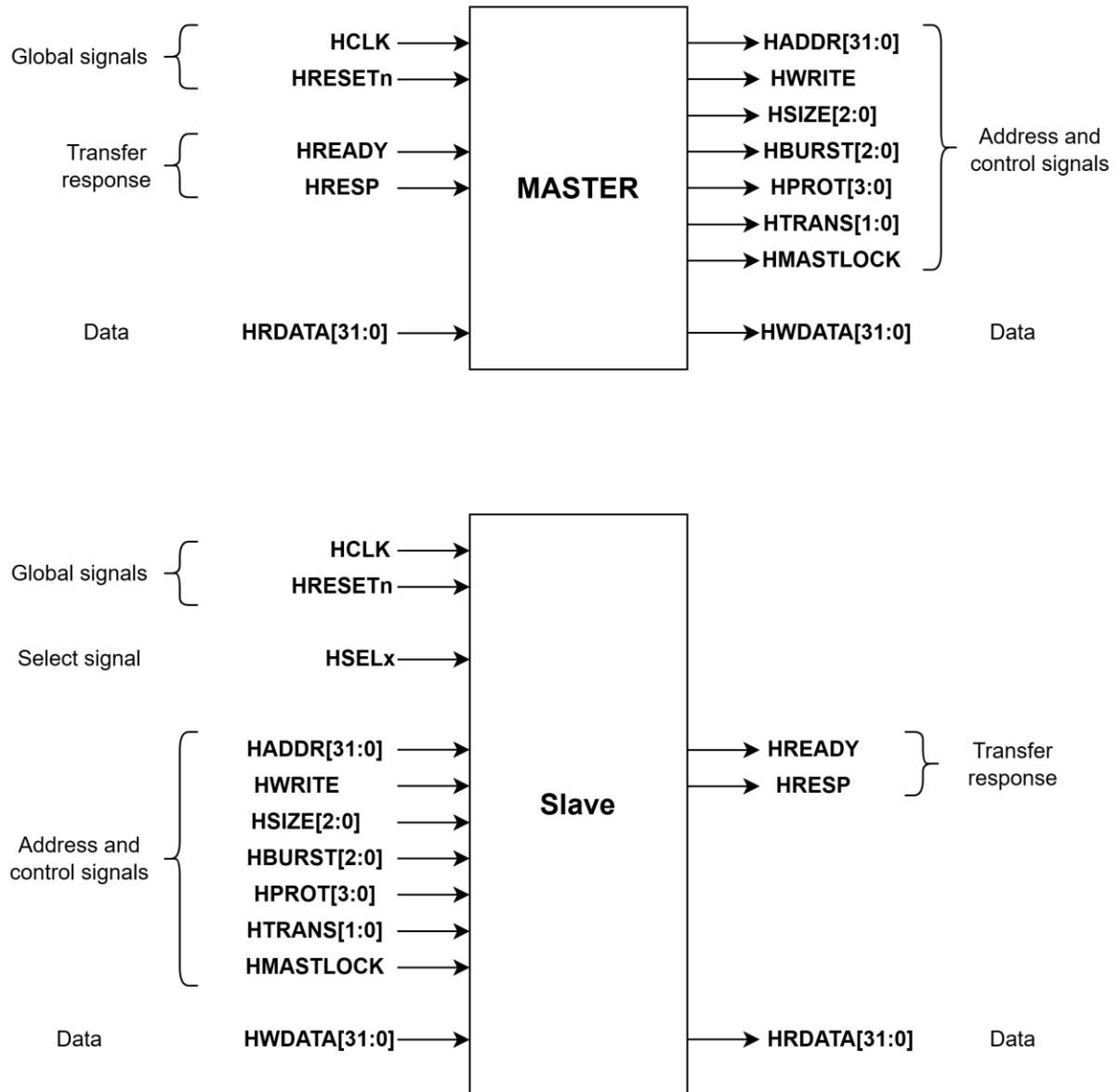


Figure 1.2. AHB Interface signals

Table 1.1. Table of AHB Interface signals

Signal group	Signal name	Direction	Size	Descriptions
Global signals	HCLK	input	1	System clock
	HRESETn	input	1	Reset signal (active low)
Master Interface Signals	HADDR	output	32	Address bus
	HWRITE	output	1	Transfer direction (1=write, 0=read)
	HSIZE	output	3	Transfer size (byte, halfword, word)

	HBURST	output	3	Burst type
	HTRANS	output	2	Transfer type
	HMASTLOCK	output	1	Locked transfer
	HWDATA	output	32	Write data bus (recommend: 32-256 bits)
Slave Interface Signals	HRDATA	output	32	Read data bus (recommend: 32-256 bits)
	HREADYOUT	output	1	Transfer completion (0: not ready, 1: ready)
	HRESP	output	1	Transfer response (0: error, 1: successful)
Decoder Signals	HSELx	output	1	Select the Subordinate.

2. DESIGN SPECIFICATIONS

2.1. Design features

2.1.1. Single transfer (without wait state)

In a simple transfer with no wait states: The Manager drives the address and control signals onto the bus after the rising edge of HCLK. The Subordinate then samples the address and control information on the next rising edge of HCLK. After the Subordinate has sampled the address and control, it can start to drive the appropriate HREADYOUT response. This response is sampled by the Manager on the third rising edge of HCLK.

2.1.2. Single transfer (with wait state)

A Subordinate can insert wait states into any transfer to enable additional time for completion. Each Subordinate has an HREADYOUT signal that it drives during the data phase of a transfer. The interconnect is responsible for combining the HREADYOUT signals from all Subordinates to generate a single HREADY signal that is used to control the overall progress.

2.1.3. Transfer types

Table 2.1. lists the transfers that can be classified into one of four types, as controlled by HTRANS [1:0].

Table 2.1. Table of Transfer type encoding

HTRANS[1:0]	Type	Descriptions
0b00	IDLE	Indicates that no data transfer is required. A Manager uses an IDLE transfer when it does not want to perform a data transfer. It is recommended that the Manager terminates a locked transfer with an IDLE transfer. Subordinates must always provide a zero wait state OKAY response to IDLE transfers and the transfer must be ignored by the Subordinate.
0b01	BUSY	The BUSY transfer type enables Managers to insert idle cycles in the middle of a burst. This transfer type indicates that the Manager is continuing with a burst but the next transfer cannot take place immediately. When a Manager uses the BUSY transfer type, the address and control signals must reflect the next transfer in the burst. Only undefined length bursts can have a BUSY transfer as the last cycle of a burst. Subordinates must always provide a zero wait state OKAY response to BUSY transfers and the transfer must be ignored by the Subordinate.
0b10	NONSEQ	Indicates a single transfer or the first transfer of a burst. The address and control signals are unrelated to the previous transfer. Single transfers on the bus are treated as bursts of length one and therefore the transfer type is NONSEQUENTIAL.
0b11	SEQ	The remaining transfers in a burst are SEQUENTIAL and the address is related to the previous transfer. The control information is identical to the previous transfer. The address is equal to the address of the previous transfer plus the transfer size, in bytes, with the transfer size being signaled by the HSIZE[2:0] signals. In the case of a wrapping burst, the address of the transfer wraps at the address boundary.

2.1.4. Transfer size

HSIZE[2:0] indicates the size of a data transfer. Table 2.2. lists the possible transfer sizes. In this design, the design just uses a maximum of 32 bits widths.

Table 2.2. Table of transfer size encoding

HSIZE[2:0]	Size (bits)	Descriptions
3'b000	8	Byte
3'b001	16	Halfword
3'b010	32	Word

3'b011	64	Doubleword
3'b100	128	4-word line
3'b101	256	8-word line
3'b110	512	-
3'b111	1024	-

Note: The transfer size set by HSIZE must be less than or equal to the width of the data bus. For example, with a 32-bit data bus, HSIZE must only use the values 0b000, 0b001, or 0b010.

The HSIZE signals have the same timing as the address bus. However, they must remain constant throughout a burst of transfer. HSIZE in conjunction with HBURST determines the address boundary for wrapping bursts.

2.1.5. Burst operation

Bursts of 4, 8, and 16-beats, undefined length bursts, and single transfers are defined in this protocol. It supports incrementing and wrapping bursts:

- Incrementing bursts to access sequential locations and the address of each transfer in the burst is an increment of the previous address.
- Wrapping bursts wrap when they cross an address boundary. The address boundary is calculated as the product of the number of beats in a burst and the size of the transfer. The number of beats are controlled by HBURST, and the transfer size is controlled by HSIZE.

For example, a four-beat wrapping burst of word (4-byte) accesses wraps at 16-byte boundaries. Therefore, if the start address of the burst is 0x34, then it consists of four transfers to addresses 0x34, 0x38, 0x3C, and 0x30.

Table 2.3. Table of burst signal encoding

HBURST[2:0]	Type	Descriptions
0b000	SINGLE	Single transfer burst
0b001	INCR	Incrementing burst of undefined length
0b010	INCR4	4-beat incrementing burst
0b011	INCR8	8-beat incrementing burst
0b100	INCR16	16-beat incrementing burst
0b101	WRAP4	4-beat wrapping burst
0b110	WRAP8	8-beat wrapping burst
0b111	WRAP16	16-beat wrapping burst

Managers must not attempt to start an incrementing burst that crosses a 1KB address boundary.

Managers can perform single transfers using either:

- SINGLE transfer bursts.
- Undefined length burst that has a burst of length one.

Note: The burst size indicates the number of beats in the burst and not the number of bytes transferred. Calculate the total amount of data transferred in a burst by multiplying the number of beats by the amount of data in each beat, as indicated by HSIZE [2:0].

All transfers in a burst must be aligned to the address boundary equal to the size of the transfer. For example, word transfers must align to word address boundaries (HADDR [1:0] = 0b00), and halfword transfers to halfword address boundaries (HADDR [0] = 0). It is recommended that the address for IDLE transfers is aligned, to avoid spurious warnings in simulation.

2.1.6. Burst termination after BUSY transfer

After a burst has started, the Manager uses BUSY transfers if it requires more time before continuing with the next transfer in the burst.

During an undefined length burst, INCR, the Manager might insert BUSY transfers and then decide that no more data transfers are required. Under these circumstances, it is acceptable for the Manager to then perform a NONSEQ or IDLE transfer that then effectively terminates the undefined length of burst.

The protocol does not permit a manager to end a burst with a BUSY transfer for fixed-length bursts of type:

- Incrementing INCR4, INCR8, and INCR16.
- Wrapping WRAP4, WRAP8, and WRAP16.

These fixed-length burst types must terminate with a SEQ transfer.

The Manager is not permitted to perform a BUSY transfer immediately after a SINGLE burst. SINGLE bursts must be followed by an IDLE transfer or a NONSEQ transfer.

2.2. AHB Master

A Master provides address and control information to initiate read and write operations. Figure 2.1. shows a Master interface.

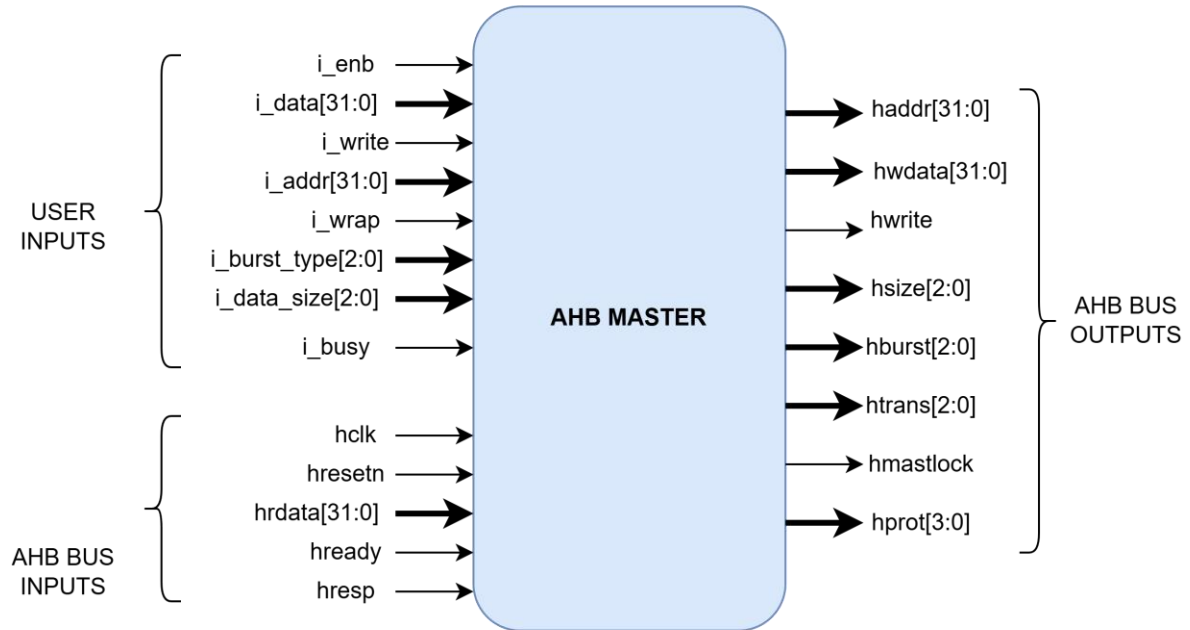


Figure 2.1. AHB Master Interface

Table 2.4. Table of AHB Master Interface signals

Port	Direction	Size	Descriptions
<code>hclk</code>	input	1	The bus clock times all bus transfers. All signal timings are related to the rising edge of <code>hclk</code> .
<code>hresetn</code>	input	1	Asynchronous reset
<code>i_enb</code>	input	1	Enable transfer signal
<code>i_data</code>	input	32	Write data to be transferred to the AHB slave.
<code>i_write</code>	input	1	Transfer direction control. A write transfer is performed when <code>i_write</code> = 1; a read transfer is performed when <code>i_write</code> = 0.
<code>i_addr</code>	input	32	Start address of the AHB transfer.
<code>i_wrap</code>	input	1	Burst wrapping control. When asserted, a WRAP burst transfer is generated.
<code>i_burst_type</code>	input	3	Burst type selection (SINGLE, INCR, WRAP4, INCR4, WRAP8, INCR8, WRAP16, INCR16).
<code>i_data_size</code>	input	3	Transfer size selection corresponding to the AHB HSIZE signal.
<code>i_busy</code>	input	1	Indicates a BUSY transfer during burst operation.

haddr	output	32	AHB address bus. Specifies the address for the current transfer.
hwdata	output	32	AHB write data bus.
hwrite	output	1	AHB transfer direction signal. High for write transfer, low for read transfer.
hsize	output	3	Indicates the size of the transfer.
hburst	output	3	AHB burst type signal. Indicates the burst mode of the current transfer.
htrans	output	2	AHB transfer type signal. Indicates IDLE, BUSY, NONSEQ, or SEQ transfer.
hrdata	input	32	AHB read data bus from the slave.
hready	input	1	When HIGH, the HREADYOUT signal indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.
hresp	input	1	The transfer response provides the Manager with additional information on the status of a transfer. When LOW, the HRESP signal indicates that the transfer status is OKAY. When HIGH, the HRESP signal indicates that the transfer status is ERROR.

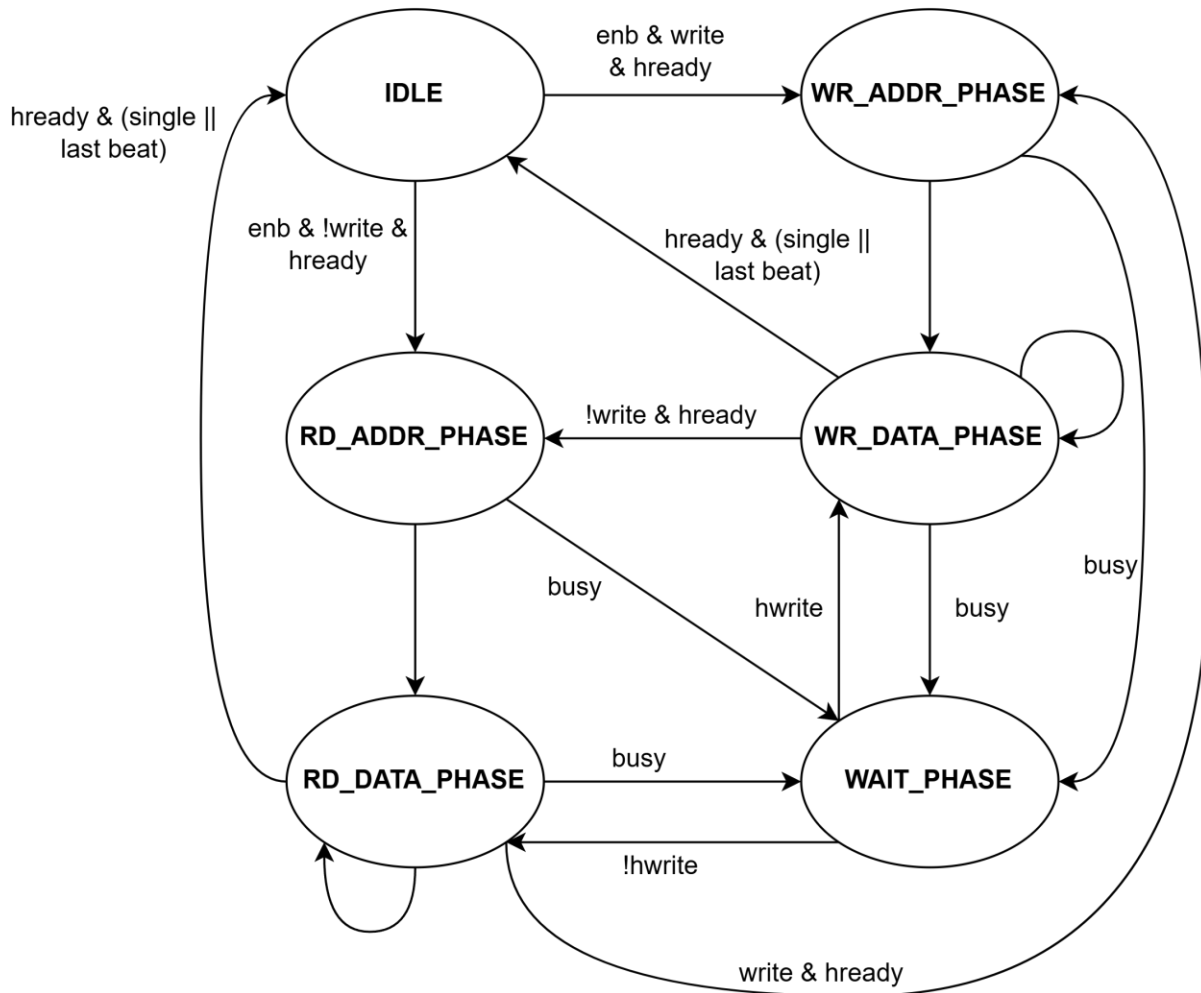


Figure 2.2. AHB Master FSM diagram

The **IDLE** state is the default state after reset and represents an inactive bus condition. In this state, no transfer is performed on the AHB bus. The controller waits for a valid transaction request while monitoring the bus ready signal.

Signal assignments:

- HTRANS = IDLE
- HWRITE = 0
- HADDR = 0
- HWDATA = 0
- HSIZE = i_data_size
- load_addr = 1

- busy_flag = 0

When i_enb is asserted and HREADY is high, the FSM enters **WR_ADDR_PHASE** for a write transaction (i_write = 1) or **RD_ADDR_PHASE** for a read transaction (i_write = 0).

The **WR_ADDR_PHASE** state corresponds to the address phase of a write transaction. During this state, the master drives the transfer address and associated control information onto the AHB bus. This state always represents the first beat of a transaction and therefore uses a non-sequential transfer type.

Signal assignments:

- HADDR = i_addr (or haddr_reg when HREADY = 0)
- HWRITE = 1
- HTRANS = NONSEQ
- HSIZE = i_data_size
- HBURST = i_burst_type
- load_addr = 0
- busy_flag = 0

After the address phase has been issued, the controller evaluates i_busy. If i_busy is deasserted, the FSM transitions to **WR_DATA_PHASE** and proceeds with the write data transfer. If i_busy is asserted, the FSM enters **WAIT_PHASE** and issues an AHB BUSY transfer while preserving all address and control information associated with the current transaction.

The **WR_DATA_PHASE** state performs the write data transfer. The write data is driven onto the HWDATA bus while maintaining the corresponding address and control information. For burst transfers, the internal address generator updates the transfer address for each beat.

Signal assignments:

- HWRITE = 1
- HSIZE = i_data_size
- HWDATA = i_data
- HBURST = i_burst_type
- load_addr = 0
- busy_flag = 0
- For a single transfer:
- HADDR = i_addr

- HTRANS = NONSEQ
For burst transfers:
- HADDR = internal_address
- HTRANS = SEQ

The FSM remains in this state until the transfer is completed. If i_busy is asserted, the controller enters **WAIT_PHASE**. For a single transfer, the FSM returns to IDLE when HREADY is asserted. For burst transfers, the FSM remains in this state until the final beat is completed.

The **RD_ADDR_PHASE** state represents the address phase of a read transaction. The master drives the target address and transfer attributes while indicating a read operation through HWRITE.

Signal assignments:

- HADDR = i_addr (or haddr_reg when HREADY = 0)
- HWRITE = 0
- HTRANS = NONSEQ
- HSIZE = i_data_size
- HBURST = i_burst_type
- load_addr = 0
- busy_flag = 0

Upon completion of the address phase, the controller checks i_busy. If i_busy is deasserted, the FSM transitions to **RD_DATA_PHASE** and waits for read data from the slave. If i_busy is asserted, the FSM enters **WAIT_PHASE** and generates an AHB BUSY transfer while retaining the current transaction context.

The **RD_DATA_PHASE** state manages the data phase of a read transaction. During burst transfers, the internal address generator provides the address for each subsequent beat while the slave returns data on the HRDATA bus.

Signal assignments:

- HWRITE = 0
- HSIZE = i_data_size
- HBURST = i_burst_type
- load_addr = 0
- busy_flag = 0

- For a single transfer:
- HADDR = i_addr
- HTRANS = NONSEQ

For burst transfers:

- HADDR = internal_address
- HTRANS = SEQ

The FSM remains in this state until the transfer is completed. When i_busy is asserted, the controller enters **WAIT_PHASE**. Upon completion of the final beat, the FSM returns to **IDLE**.

The **WAIT_PHASE** state is used to generate an AHB BUSY transfer. This state temporarily suspends an ongoing burst transaction while preserving all transaction information.

Signal assignments:

- HADDR = haddr_reg
- HWDATA = hwdata_reg
- HWRITE = hwrite_reg
- HSIZE = hsize_reg
- HBURST = hburst_reg
- HTRANS = BUSY
- load_addr = 1
- busy_flag = 1

All transaction-related outputs are held constant during this state. When i_busy is deasserted, the FSM resumes the interrupted transaction by returning to either **WR_DATA_PHASE** or **RD_DATA_PHASE** depending on the current transfer direction.

2.3. AHB Slave

A Subordinate responds to transfers initiated by Managers in the system. The Subordinate uses the HSELx select signal from the decoder to control when it responds to a bus transfer.

The Subordinate signals back to the Manager:

- The completion or extension of the bus transfer.
- The success or failure of the bus transfer.

Figure 2.3. shows a Slave interface.

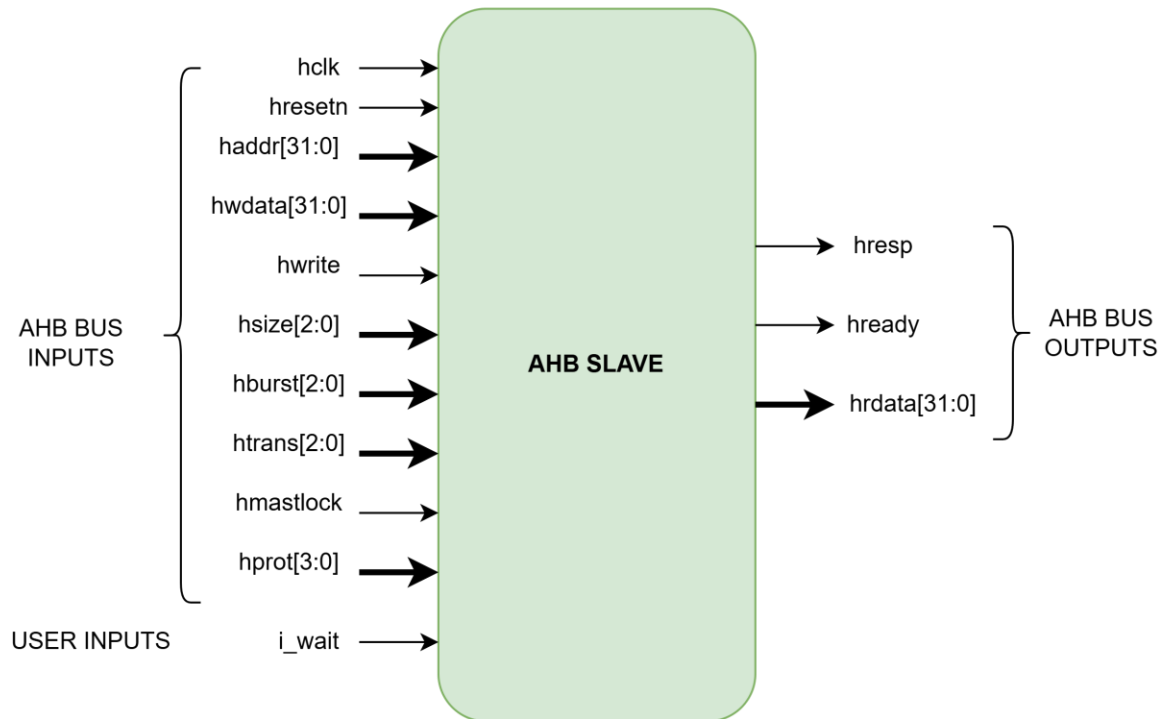


Figure 2.3. AHB Slave Interface

Table 2.4. Table of AHB Slave Interface signals

Port	Direction	Size	Descriptions
hclk	input	1	The bus clock times all bus transfers. All signal timings are related to the rising edge of hclk.
hresetn	input	1	Asynchronous reset
haddr	input	32	AHB address bus. Specifies the address for the current transfer.
hwdata	input	32	AHB write data bus.
hwrite	input	1	AHB transfer direction signal. High for write transfer, low for read transfer.
hsize	input	3	Indicates the size of the transfer.
hburst	input	3	AHB burst type signal. Indicates the burst mode of the current transfer.
htrans	input	2	AHB transfer type signal. Indicates IDLE, BUSY, NONSEQ, or SEQ transfer.
hrdata	output	32	AHB read data bus from the slave.
hready	output	1	When HIGH, the HREADYOUT signal indicates that a transfer has finished on the bus. This signal can be driven LOW to extend a transfer.
hresp	output	1	The transfer response provides the Manager with additional information on the status of a

			transfer. When LOW, the HRESP signal indicates that the transfer status is OKAY. When HIGH, the HRESP signal indicates that the transfer status is ERROR.
--	--	--	---

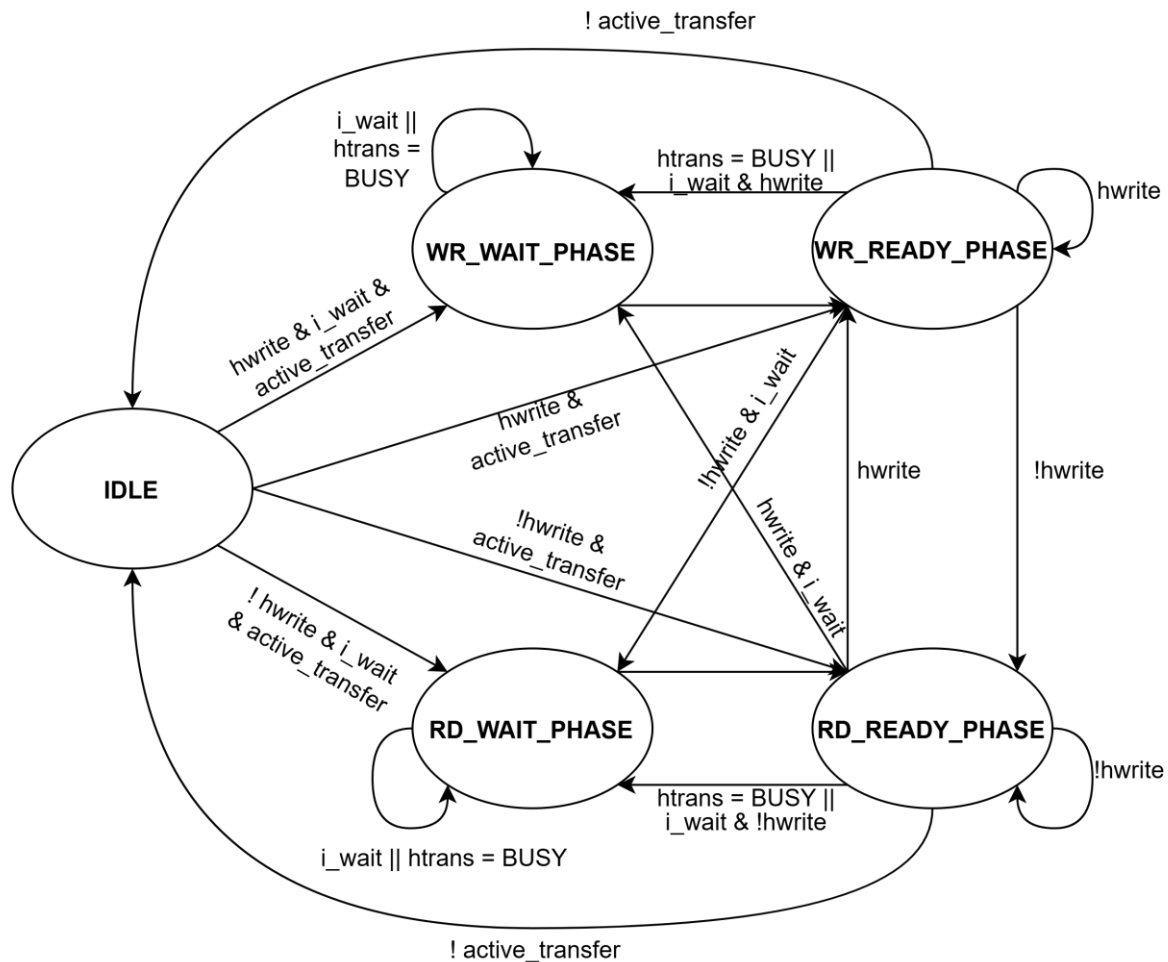


Figure 2.4. AHB Slave FSM diagram

The AHB Slave is controlled by a finite state machine (FSM) consisting of five states: **IDLE**, **WR_READY_PHASE**, **WR_WAIT_PHASE**, **RD_READY_PHASE**, and **RD_WAIT_PHASE**. The FSM monitors the AHB control signals and generates the appropriate slave response signals, including HREADY, HRESP, and memory access control signals. The state machine supports both read and write transactions and provides wait-state insertion through the *i_wait* signal.

A valid transfer is detected using the signal:

active_transfer = hsel & ((htrans == NONSEQ) || (htrans == SEQ));

which indicates that the slave is selected and an active AHB transfer is in progress.

The **IDLE** state represents the inactive condition of the slave. In this state, the slave waits for a valid AHB transaction. When an active write transfer is detected, the FSM transitions to either **WR_READY_PHASE** or **WR_WAIT_PHASE** depending on the value of *i_wait*. Similarly, when an active read transfer is detected, the FSM transitions to either **RD_READY_PHASE** or **RD_WAIT_PHASE**. If no valid transfer is present, the FSM remains in the **IDLE** state.

Signal assignments:

- **HREADY = 1**
- **HRESP = OKAY (0)**
- **write_en = 0**
- **ahb_read = 0**

The **WR_READY_PHASE** state handles write transactions when the slave is ready to accept data. During this state, the write enable signal is asserted, allowing the incoming write data to be stored in the slave memory. The slave indicates successful completion of the transfer by asserting **HREADY**.

Back-to-back write transactions are supported by remaining in the same state when another valid write transfer is detected. If the next transaction is a read transfer, the FSM transitions directly to the read path without returning to **IDLE**. When a **BUSY** transfer is detected or a wait state is requested through *i_wait*, the FSM enters the **WR_WAIT_PHASE** state.

Signal assignments:

- **HREADY = 1**
- **HRESP = OKAY (0)**
- **write_en = 1**

- ahb_read = 0

The **WR_WAIT_PHASE** state inserts wait cycles during a write transaction. While the slave remains in this state, the current transaction is stalled and the master is prevented from proceeding to the next transfer.

The FSM stays in this state as long as i_wait remains asserted or the transfer type is BUSY. Once both conditions are cleared, the FSM returns to **WR_READY_PHASE** to continue the pending write transaction.

Signal assignments:

- HREADY = 0
- HRESP = OKAY (0)
- write_en = 0
- ahb_read = 0

The **RD_READY_PHASE** state handles read transactions when the slave is able to provide data immediately. During this state, the read control signal is asserted and the requested data is driven onto the HRDATA bus through the internal memory read logic.

Back-to-back read transfers are supported by remaining in the same state when a subsequent read transaction is detected. The FSM can also switch directly from a read transaction to a write transaction without returning to **IDLE**. If a wait state is required or a BUSY transfer is detected, the FSM transitions to **RD_WAIT_PHASE**.

Signal assignments:

- HREADY = 1 when active_transfer = 1
- HRESP = OKAY (0)
- write_en = 0
- ahb_read = 1

The **RD_WAIT_PHASE** state inserts wait cycles during a read transaction. This state is typically used when the requested data is not immediately available or when the slave needs additional cycles before completing the read operation.

The slave deasserts HREADY, causing the master to stall and maintain the current transfer. The FSM remains in this state while i_wait is asserted or while a BUSY transfer is present. Once the wait condition is removed, the controller returns to **RD_READY_PHASE** and completes the read operation.

Signal assignments:

- HREADY = 0
- HRESP = OKAY (0)
- write_en = 0
- ahb_read = 0

2.4. Address Decoder

This component decodes the address of each transfer and provides a select signal for the Subordinate that is involved in the transfer. It also provides a control signal to the multiplexor. Figure 2.5. shows the address structure.

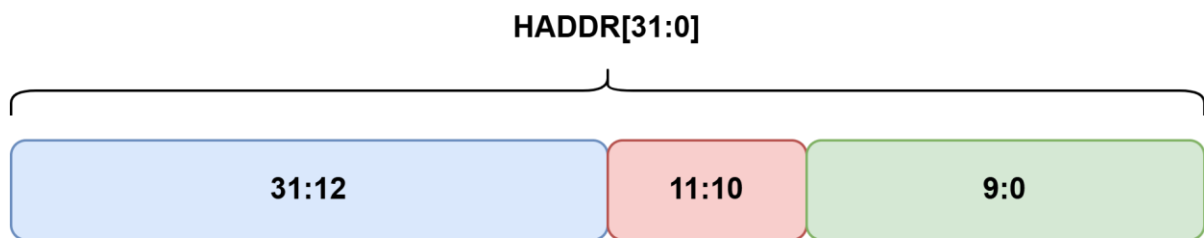


Figure 2.5. Address Configuration

The AHB Address Decoder is responsible for selecting the target slave based on the address issued by the AHB master. The decoder monitors the address bus (HADDR) and transfer type (HTRANS) and generates individual slave select signals (HSELx) together with a multiplexer selection signal (mux_select). This mechanism ensures that only one slave is selected for a valid transfer and that the corresponding slave response is routed back to the master. A transfer is considered valid when the transfer type is different from IDLE:

transfer_valid = (HTRANS != IDLE)
--

Only valid transfers participate in the address decoding process. When the bus is idle, all slave select signals are deasserted.

The decoder divides the 32-bit address into three fields as shown in Figure X. Address bits [11:10] are used as the slave selection field, while bits [9:0] represent the local address space within each slave. The upper address bits [31:12] are currently ignored by the decoder and may be reserved for future system expansion.

Table 2.5. Table of Address Slave selection configuration

HADDR[11:10]	Selected Slave	HSEL Signal	mux_select
2'b00	Slave 1	hsel_1	2'b00
2'b01	Slave 2	hsel_2	2'b01
2'b10	Slave 3	hsel_3	2'b10
2'b11	Slave 4	hsel_4	2'b11

For every valid transfer, the decoder examines HADDR[11:10] and asserts exactly one slave select output. The corresponding multiplexer selection value is generated simultaneously to route the selected slave's read data and response signals back to the master.

From Figure 2.5. and Table 2.5. we have Table 2.6. which describes the base address for each slave.

Table 2.6. Table of base address for each slave

Slave	Base address
Slave 1	32'h0000_0000
Slave 2	32'h0000_0400
Slave 3	32'h0000_0800
Slave 4	32'h0000_0C00

3. AHB SYSTEM OPERATION

The Manager starts a transfer by driving the address and control signals. These signals provide information about the address, direction, width of the transfer, and indicate if the transfer forms part of a burst. Transfers can be: Single.

- Incrementing bursts that do not wrap at address boundaries.
- Wrapping bursts that wrap at particular address boundaries.
- The write data bus moves data from the Manager to a Subordinate, and the read data bus moves data from a Subordinate to the Manager.

Every transfer consists of:

- Address phase One address and control cycle.
- Data phase One or more cycles for the data.

A Subordinate cannot request that the address phase is extended and therefore all Subordinates must be capable of sampling the address during this time. However, a Subordinate can request that the Manager extends the data phase by using HREADY. This signal, when LOW, causes wait states to be inserted into the transfer and enables the Subordinate to have extra time to provide or sample data.

The Subordinate uses HRESP to indicate the success or failure of a transfer.

3.1. Single transfer (Write & Read)

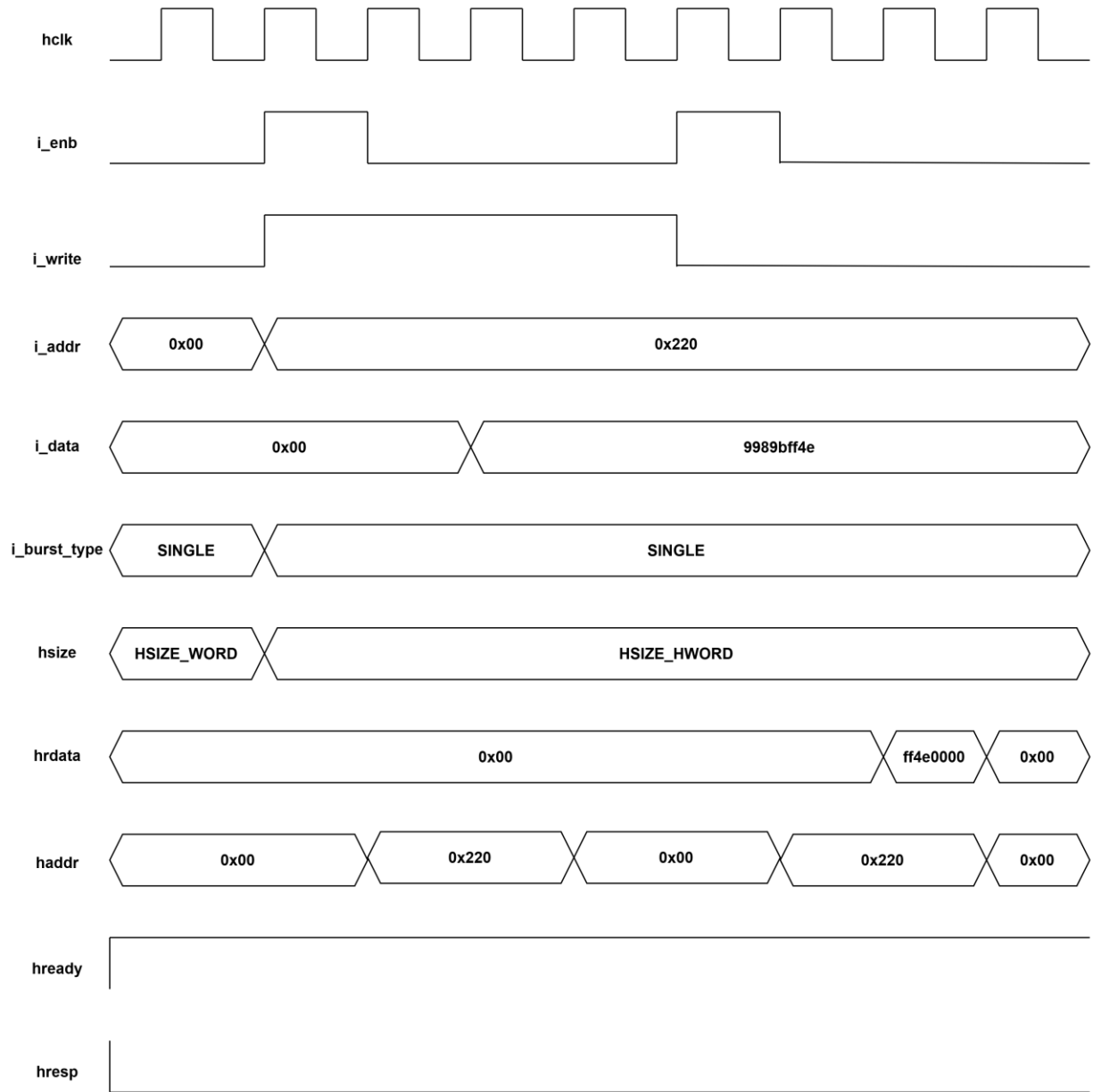


Figure 3.1. Single transfer waveform

Idle Phase

- HSEL = 0, indicating that the interface is idle, and no transfer is active.

Address Phase – Transfer 1

- HSEL is asserted, indicating that the Manager is starting a new transfer.
- Transfer information, including HADDR, HBURST, and HTRANS, is driven onto the bus.
- The address and control signals are sampled on the rising edge of the HCLK.

Data Phase – Transfer 1

- HREADY = 1 throughout the transfer, indicating that the Slave is ready.
- The transfer completes immediately without inserting any wait states.

Address Phase – Transfer 2

- A subsequent transfer is initiated with HTRANS = 0.
- HADDR remains unchanged from the previous transfer.

Data Phase – Transfer 2

- This is the data phase of the read transfer.
- The read data HRDATA appears on the bus and is sampled when HREADY = 1.
- HRESP = 0 throughout the waveform, indicating that no error response occurs.

3.2. Address Decode transfer

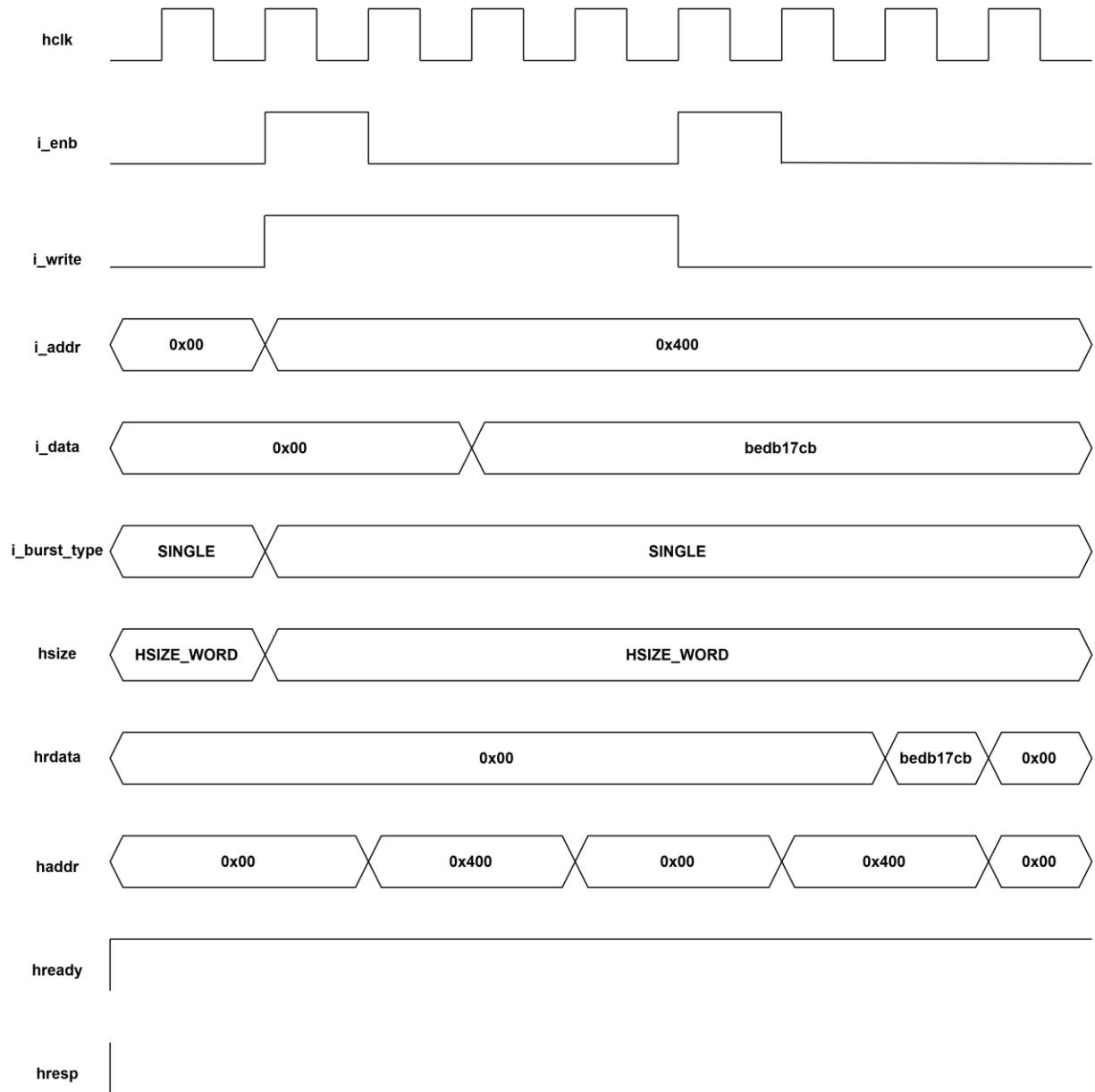


Figure 3.2. Address decoder for Slave 2

Idle Phase

- The interface starts in the idle state with the initial bus values set to 0x00.

Address Phase – Transfer 1

- HADDR changes from 0x00 to 0x400, indicating a new transfer to address 0x400.
- HBURST = SINGLE, so the transfer is a single-beat transaction.
- HSIZE = HSIZE_WORD, indicating a word-sized transfer.

Data Phase – Transfer 1

- The corresponding data value is 0xbedb17cb.
- The transfer completes without additional wait cycles.

Address Phase – Transfer 2

- A subsequent transfer is initiated with the address remaining at 0x400.
- The transfer type remains SINGLE with HSIZE_WORD.

Data Phase – Transfer 2

- The data value 0xbedb17cb appears during the data phase.
- This indicates that the transfer returns the expected word data.

Subsequent Transfers

- The waveform shows alternating address/data values between 0x00 and 0x400.
- The transfers continue as single word-sized transactions, with no visible wait-state extension.

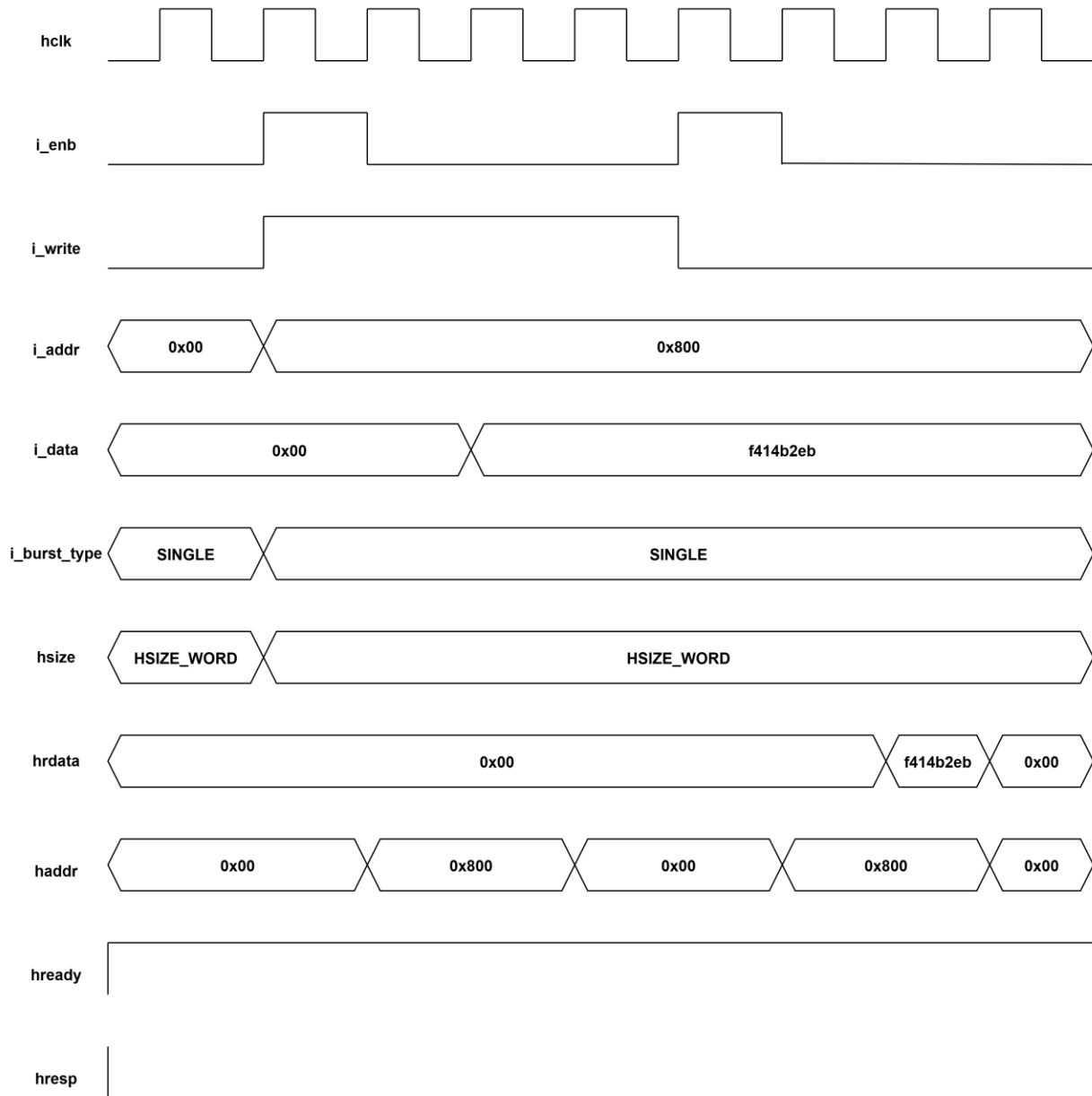


Figure 3.3. Address decoder for Slave 3

Idle Phase

- The interface starts in the idle state with the initial bus values set to 0x00.

Address Phase – Transfer 1

- HADDR changes from 0x00 to 0x800, indicating a new transfer to address 0x800 (choose Slave 3).
- HBURST = SINGLE, so the transfer is a single-beat transaction.
- HSIZE = HSIZE_WORD, indicating a word-sized transfer.

Data Phase – Transfer 1

- The corresponding data value is 0xf414b2eb.
- The transfer completes without additional wait cycles.

Address Phase – Transfer 2

- A subsequent transfer is initiated with the address remaining at 0x800 (choose Slave 3).
- The transfer type remains SINGLE with HSIZE_WORD.

Data Phase – Transfer 2

- The data value 0xf414b2eb appears during the data phase.
- This indicates that the transfer returns the expected word data.

Subsequent Transfers

- The waveform shows alternating address/data values between 0x00 and 0x800.
- The transfers continue as single word-sized transactions, with no visible wait-state extension.

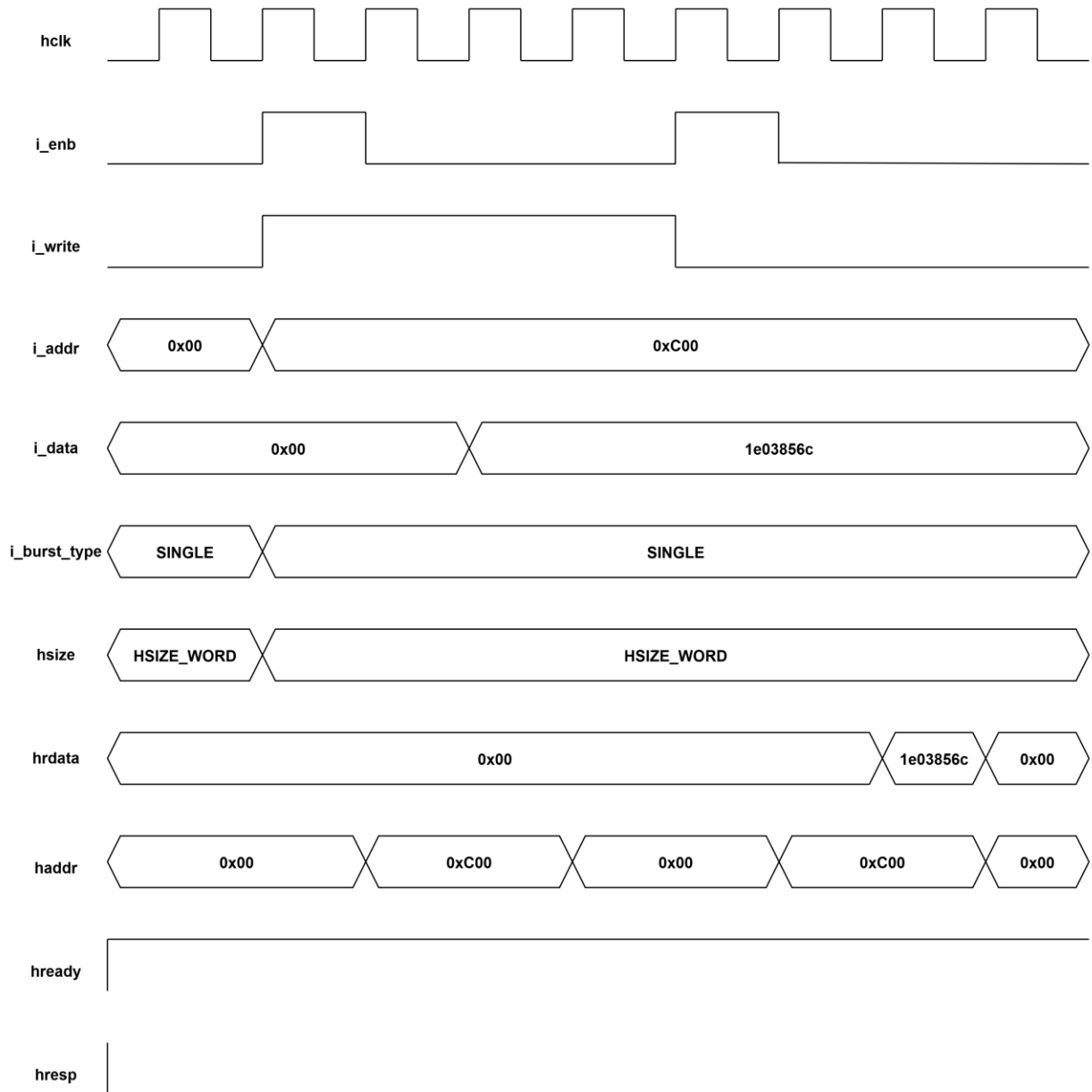


Figure 3.4. Address decoder for Slave 4

Idle Phase

- The interface starts in the idle state with the initial bus values set to 0x00.

Address Phase – Transfer 1

- HADDR changes from 0x00 to 0xC00, indicating a new transfer to address 0xC00 (choose Slave 4).
- HBURST = SINGLE, so the transfer is a single-beat transaction.
- HSIZE = HSIZE_WORD, indicating a word-sized transfer.

Data Phase – Transfer 1

- The corresponding data value is 0x1e03856c.
- The transfer completes without additional wait cycles.

Address Phase – Transfer 2

- A subsequent transfer is initiated with the address remaining at 0xC00 (choose Slave 4).
- The transfer type remains SINGLE with HSIZE_WORD.

Data Phase – Transfer 2

- The data value 0x1e03856c appears during the data phase.
- This indicates that the transfer returns the expected word data.

Subsequent Transfers

- The waveform shows alternating address/data values between 0x00 and 0xC00.
- The transfers continue as single word-sized transactions, with no visible wait-state extension.

3.3. Master waiting transfer

The Master waiting transfer is the transfer that when we assert `i_busy` the transfer type is BUSY. The example and waveform is shown as part 3.7.

3.4. Slave waiting transfer

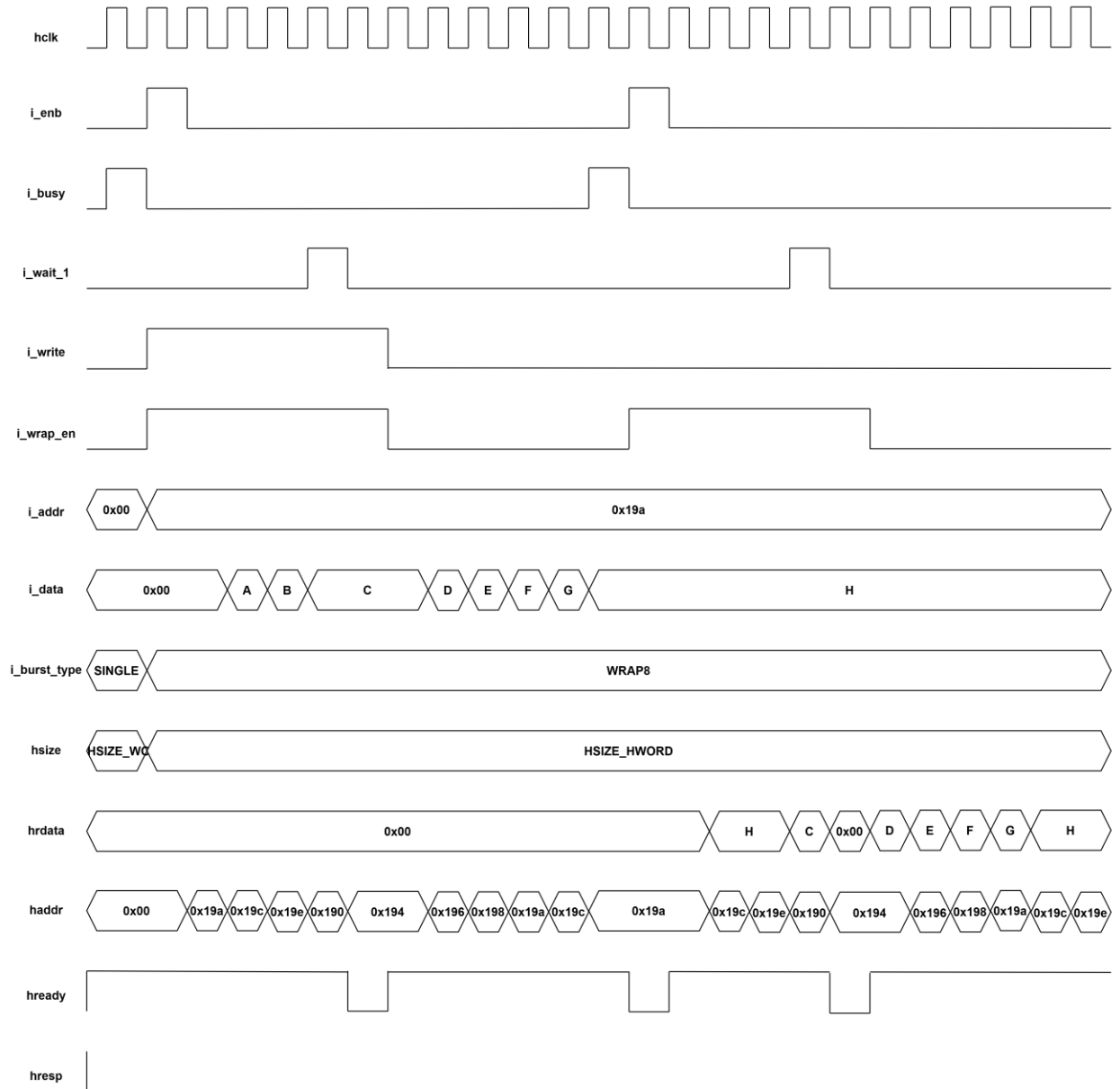


Figure 3.5. Waveform of Slave waiting transfer

Idle Phase

- The interface starts in the idle state with $i_enb = 0$ and the initial address set to 0x00.
- $hready$ remains high and $hresp$ remains low.

Address Phase – Burst Start

- i_addr changes from 0x00 to 0x19A, indicating the start address of the burst transfer.
- $i_burst_type = WRAP8$, indicating an 8-beat wrapping burst.

- `hsize = HSIZE_HWORD`, so each transfer occupies 2 bytes.
- The starting address `0x19A` is driven onto `haddr`.

Data Phase

- The transferred data progresses through A, B, C, D, E, F, G, and H.
- During the read transfer, `hrdata` returns H, C, D, E, F, G, H, and H as shown in the waveform.
- `i_wait_1` is asserted, causing Slave 1 to enter the wait state.
- While Slave 1 is waiting, `hready` is driven low and the current transfer is extended.
- When `hready` returns high, the burst transfer resumes normally.

Address Progression

- Since `HSIZE_HWORD` is selected, the address increases by 2 bytes for each beat.
- The address sequence is: `0x19A → 0x19C → 0x19E → 0x190 → 0x192 → 0x194 → 0x196 → 0x198`.
- The burst wraps within the 16-byte boundary `0x190–0x19F`.
- After reaching `0x19E`, the next address wraps to `0x190` instead of continuing to `0x1A0`.
- When `i_wait_1` is asserted, `haddr` remains unchanged because the current transfer has not yet completed.
- Address progression resumes only after `hready` returns high.
- The waveform shows the wrapped sequence continuing within the same boundary until all eight beats are completed.

Slave Waiting Behavior

- `i_wait_1` is used to place Slave 1 into a waiting state.
- When `i_wait_1` is asserted, Slave 1 deasserts `hready` to indicate that it is not ready to complete the current transfer.
- The master must hold the current address, control signals, and transfer information stable while `hready` remains low.
- No new transfer is accepted during the wait state.
- Once Slave 1 completes its internal operation, `hready` is asserted high and the burst continues from the stalled transfer.

Burst Behavior

- `WRAP8` defines an eight-beat wrapping burst.

- The waveform shows consecutive halfword transfers with wrapping addresses.
- The address sequence follows the wrap pattern: 0x19A → 0x19C → 0x19E → 0x190 → 0x192 → 0x194 → 0x196 → 0x198.
- The wrap boundary size is 16 bytes (8 beats × 2 bytes per beat).
- A wait state is inserted when i_wait_1 is asserted and hready goes low.
- The burst resumes once Slave 1 asserts hready high again.
- hresp = 0 throughout the waveform, indicating that the transfer completes without an error response.

3.5. INCR burst transfer

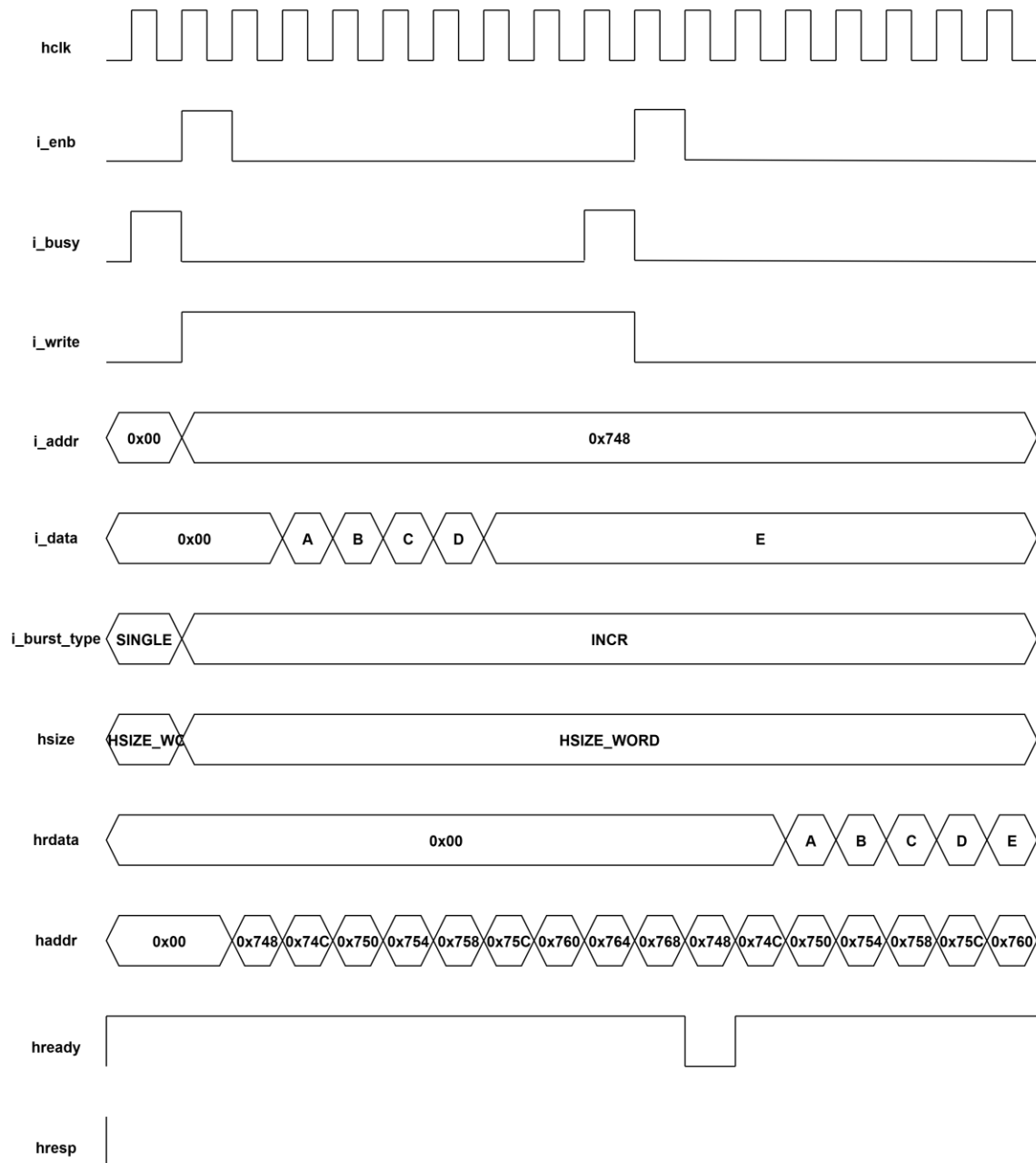


Figure 3.6 Waveform of INCR burst transfer

Idle Phase

- The interface starts in the idle state with the initial address set to 0x00.

Address Phase – Burst Start

- HADDR changes to 0x748, indicating the start address of the burst transfer.
- HBURST = INCR, indicating an incrementing burst with an unspecified number of beats.
- HSIZE = HSIZE_WORD, so each transfer is one word (4 bytes).

Data Phase

- The transferred data progresses through A, B, C, D, and E.
- Each data beat corresponds to an incremented address in the burst.

Address Progression

- The address increases by 4 bytes per beat:
- $0x748 \rightarrow 0x74C \rightarrow 0x750 \rightarrow 0x754 \rightarrow 0x758 \rightarrow 0x75C \rightarrow 0x760 \rightarrow 0x764 \rightarrow 0x768$
- The burst then starts another sequence from $0x748$.

Burst Behavior

- INCR allows the burst to continue for an arbitrary number of transfers.
- The waveform demonstrates consecutive word transfers with automatically incremented addresses.

3.6. INCR4/INCR8/INCR16 burst transfer

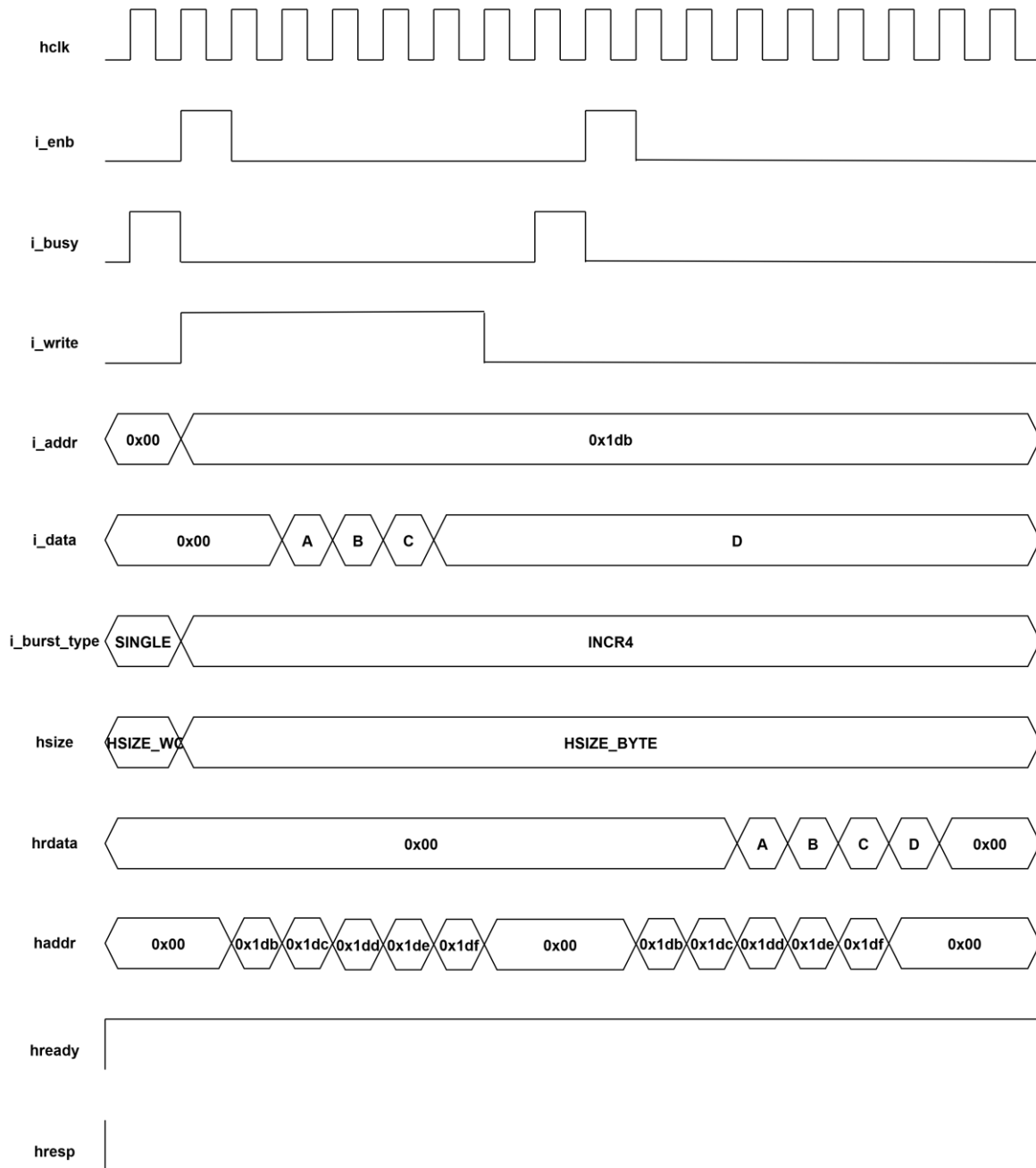


Figure 3.7. Waveform of INCR4 burst transfer

Idle Phase

- The interface starts in the idle state with $i_enb = 0$ and the initial address set to 0x00.
- $hready$ remains high and $hresp$ remains low.

Address Phase – Burst Start

- i_addr changes from 0x00 to 0x1DB, indicating the start address of the burst transfer.
- i_burst_type = INCR4, indicating a 4-beat incrementing burst.
- hsize = HSIZE_BYTE, so each transfer is one byte.
- The starting address 0x1DB is driven onto haddr.

Data Phase

- The transferred data progresses through A, B, C, and D.
- During the read transfer, hrdata returns A, B, C, and D.
- hready remains high, so no wait state is inserted.

Address Progression

- Since HSIZE_BYTE is selected, the address increases by 1 byte for each beat.
- The address sequence is: 0x1DB → 0x1DC → 0x1DD → 0x1DE.
- The waveform also shows 0x1DF following the four burst addresses before haddr returns to 0x00.
- A subsequent sequence starts again from 0x1DB.

Burst Behavior

- INCR4 defines a four-beat incrementing burst.
- The waveform shows consecutive byte transfers with incrementing addresses.
- hready = 1 throughout the transfer, indicating no wait states.
- hresp = 0 throughout the waveform, indicating no error response is shown.

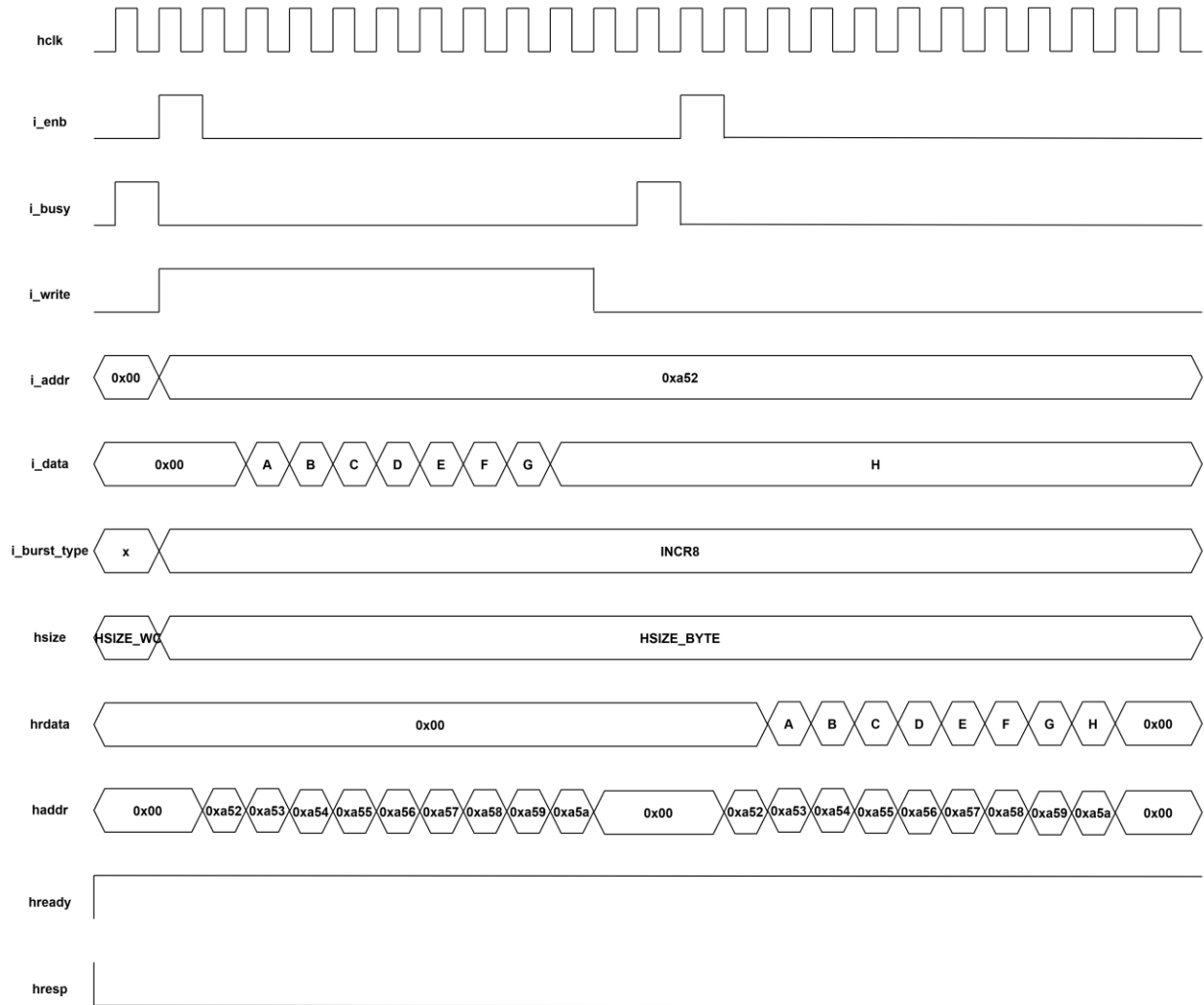


Figure 3.8. Waveform of INCR8 burst transfer

Idle Phase

- The interface starts in the idle state with $i_enb = 0$ and the initial address set to 0x00.
- $hready$ remains high and $hresp$ remains low.

Address Phase – Burst Start

- i_addr changes from 0x00 to 0xA52, indicating the start address of the burst transfer.
- $i_burst_type = INCR8$, indicating a 8-beat incrementing burst.
- $hsize = HSIZE_BYTE$, so each transfer is one byte.
- The starting address 0xA52 is driven onto $haddr$.

Data Phase

- The transferred data progresses through A, B, C, D, E, F, G, and H.

- During the read transfer, hrdata returns A, B, C, D, E, F, G, and H.
- hready remains high, so no wait state is inserted.

Address Progression

- Since HSIZE_BYTE is selected, the address increases by 1 byte for each beat.
- The address sequence is: 0xA52 → 0xA53 → 0xA54 → 0xA55 → 0xA56 → 0xA57 → 0xA58 → 0xA59 → 0xA5A.
- The waveform also shows 0xA5A following the four burst addresses before haddr returns to 0x00.
- A subsequent sequence starts again from 0xA52.

Burst Behavior

- INCR8 defines a four-beat incrementing burst.
- The waveform shows consecutive byte transfers with incrementing addresses.
- hready = 1 throughout the transfer, indicating no wait states.
- hresp = 0 throughout the waveform, indicating no error response is shown.

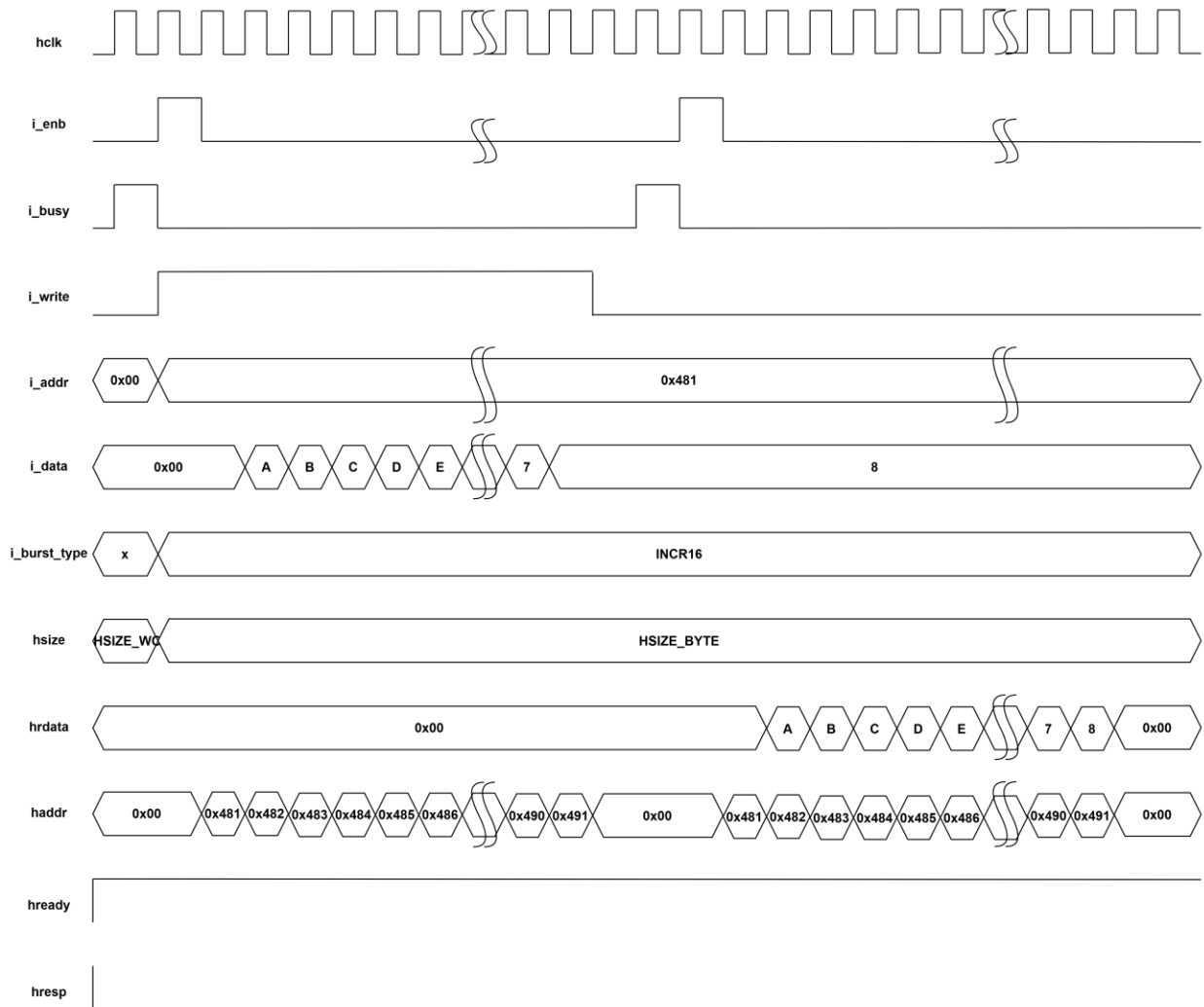


Figure 3.9 Waveform of INCR16 burst transfer

Idle Phase

- The interface starts in the idle state with $i_enb = 0$ and the initial address set to 0x00.
- $hready$ remains high and $hresp$ remains low.

Address Phase – Burst Start

- i_addr changes from 0x00 to 0x481, indicating the start address of the burst transfer.
- $i_burst_type = INCR16$, indicating a 16-beat incrementing burst.
- $hsize = HSIZE_BYTE$, so each transfer is one byte.
- The starting address 0x481 is driven onto $haddr$.

Data Phase

- The transferred data progresses through A, B, C, D, E, F, G, H, 1, 2, 3, 4, 5, 6, 7, and 8.
- During the read transfer, hrddata returns A, B, C, D, E, F, G, H, 1, 2, 3, 4, 5, 6, 7, and 8.
- hready remains high, so no wait state is inserted.

Address Progression

- Since HSIZE_BYTE is selected, the address increases by 1 byte for each beat.
- The address sequence is: 0x481 → 0x482 → ... → 0x490 → 0x491.
- The waveform also shows 0x491 following the four burst addresses before haddr returns to 0x00.
- A subsequent sequence starts again from 0x481.

Burst Behavior

- INCR16 defines a four-beat incrementing burst.
- The waveform shows consecutive byte transfers with incrementing addresses.
- hready = 1 throughout the transfer, indicating no wait states.
- hresp = 0 throughout the waveform, indicating no error response is shown.

3.7. WRAP4/WRAP8/WRAP16 transfer

Wrap Address Calculation

- For WRAP bursts, the address wraps within a fixed address boundary determined by the burst length and transfer size.
- The wrap boundary size is calculated as:

$\text{Wrap Size} = \text{Burst Length} \times \text{Transfer Size}$
--

- The burst length is selected according to the burst type:
 - WRAP4 = 4 beats
 - WRAP8 = 8 beats
 - WRAP16 = 16 beats
- The transfer size is determined by HSIZE:
 - HSIZE_BYTE = 1 byte
 - HSIZE_HWORD = 2 bytes
 - HSIZE_WORD = 4 bytes
 - HSIZE_DWORD = 8 bytes
 - HSIZE_128BIT = 16 bytes

Wrap Boundary Calculation

- The wrap mask is generated using the burst length and transfer size.
- For WRAP4, WRAP8, and WRAP16, the wrap mask is calculated as:

$$\text{Wrap Mask} = \text{Wrap Size} - 1$$

- The wrap base address is obtained by aligning the starting address to the wrap boundary.
- The wrap base is calculated as:

$$\text{Wrap Base} = \text{Start Address} \& \sim(\text{Wrap Mask})$$

- This wrap base defines the lower boundary of the wrapping region.

Address Progression

- During each valid transfer (hready = 1), the address is incremented by the transfer size.
- The next address is generated using:

$$\text{Next Address} = \text{Wrap Base} | ((\text{Current Address} + \text{Transfer Size}) \& \text{Wrap Mask})$$

- If the incremented address remains within the wrap boundary, the address increases normally.
- If the incremented address exceeds the upper boundary, the address wraps back to the lower boundary.
- This ensures that all transfers remain within the same wrapping region.

Example: WRAP16 with HSIZE_WORD

- The burst type is WRAP16, corresponding to sixteen transfers.
- HSIZE_WORD is selected, so each transfer occupies 4 bytes.
- The wrap size is:

$$\text{Wrap Size} = 16 \times 4 = 64 \text{ bytes}$$

- For a starting address of 0x4A4:

$$\text{Wrap Mask} = 0x3F$$

Wrap Base = 0x4A4 & ~0x3F = 0x480

- The wrapping region is therefore 0x480–0x4BF.
- The generated address sequence is:

0x4A4 → 0x4A8 → 0x4AC → 0x4B0 → 0x4B4 → 0x4B8 → 0x4BC → 0x480 → 0x484 → 0x488 → 0x48C → 0x490 → 0x494 → 0x498 → 0x49C → 0x4A0

- After reaching 0x4BC, the next increment would produce 0x4C0.
- Since 0x4C0 exceeds the upper boundary of the wrapping region, the address wraps back to 0x480.
- The waveform therefore shows a continuous WRAP16 burst operating within a fixed 64-byte address boundary.

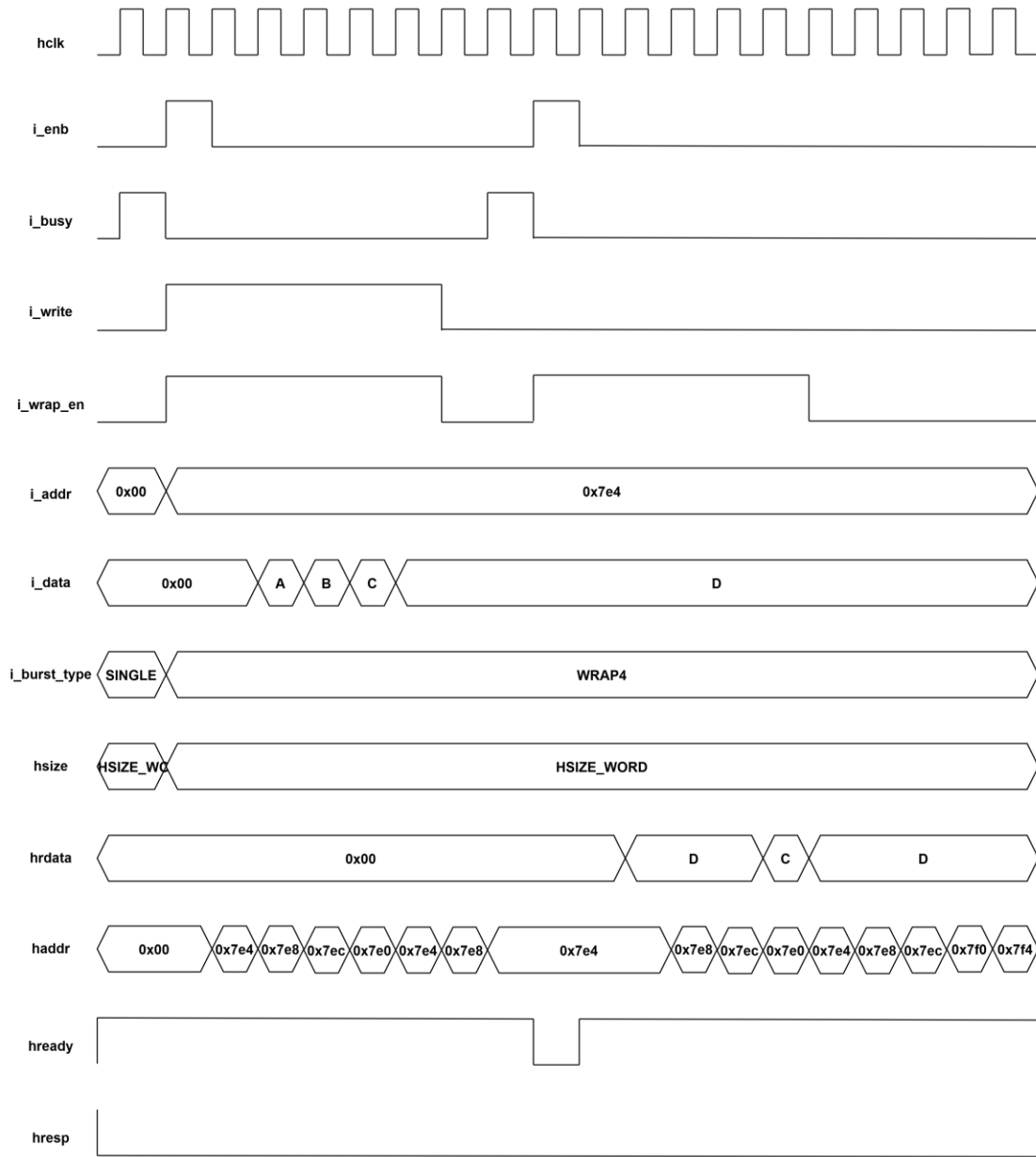


Figure 3.10. Waveform of WRAP4 burst transfer

Idle Phase

- The interface starts in the idle state with $i_enb = 0$ and the initial address set to 0x00.
- $hready$ remains high and $hresp$ remains low.

Address Phase – Burst Start

- `i_addr` changes from 0x00 to 0x7E4, indicating the start address of the burst transfer.
- `i_burst_type` = WRAP4, indicating a 4-beat wrapping burst.
- `hsize` = HSIZE_WORD, so each transfer is one word (4 bytes).
- The starting address 0x7E4 is driven onto `haddr`.

Data Phase

- The transferred data progresses through A, B, C, and D.
- During the read transfer, `hrdata` returns D, C, and D as shown in the waveform.
- When `i_busy` is asserted, `hready` is driven low, inserting a wait state into the burst transfer.
- While `hready` is low, the current transfer is extended and the address/control information remains unchanged until the transfer completes.

Address Progression

- Since HSIZE_WORD is selected, the address increases by 4 bytes for each beat.
- The address sequence is: 0x7E4 → 0x7E8 → 0x7EC → 0x7E0.
- The burst wraps within the 16-byte boundary 0x7E0–0x7EF.
- After reaching 0x7EC, the next address wraps to 0x7E0 instead of continuing to 0x7F0.
- When `hready` is deasserted due to `i_busy`, address progression is temporarily stalled until `hready` returns high.
- A subsequent sequence starts again from 0x7E4 as shown in the waveform.

Burst Behavior

- WRAP4 defines a four-beat wrapping burst.
- The waveform shows consecutive word transfers with wrapping addresses.
- The address sequence follows the wrap pattern: 0x7E4 → 0x7E8 → 0x7EC → 0x7E0.
- A wait state is inserted when `i_busy` is asserted and `hready` goes low.
- The burst resumes once `hready` returns high.
- `hresp` = 0 throughout the waveform, indicating no error response is shown.

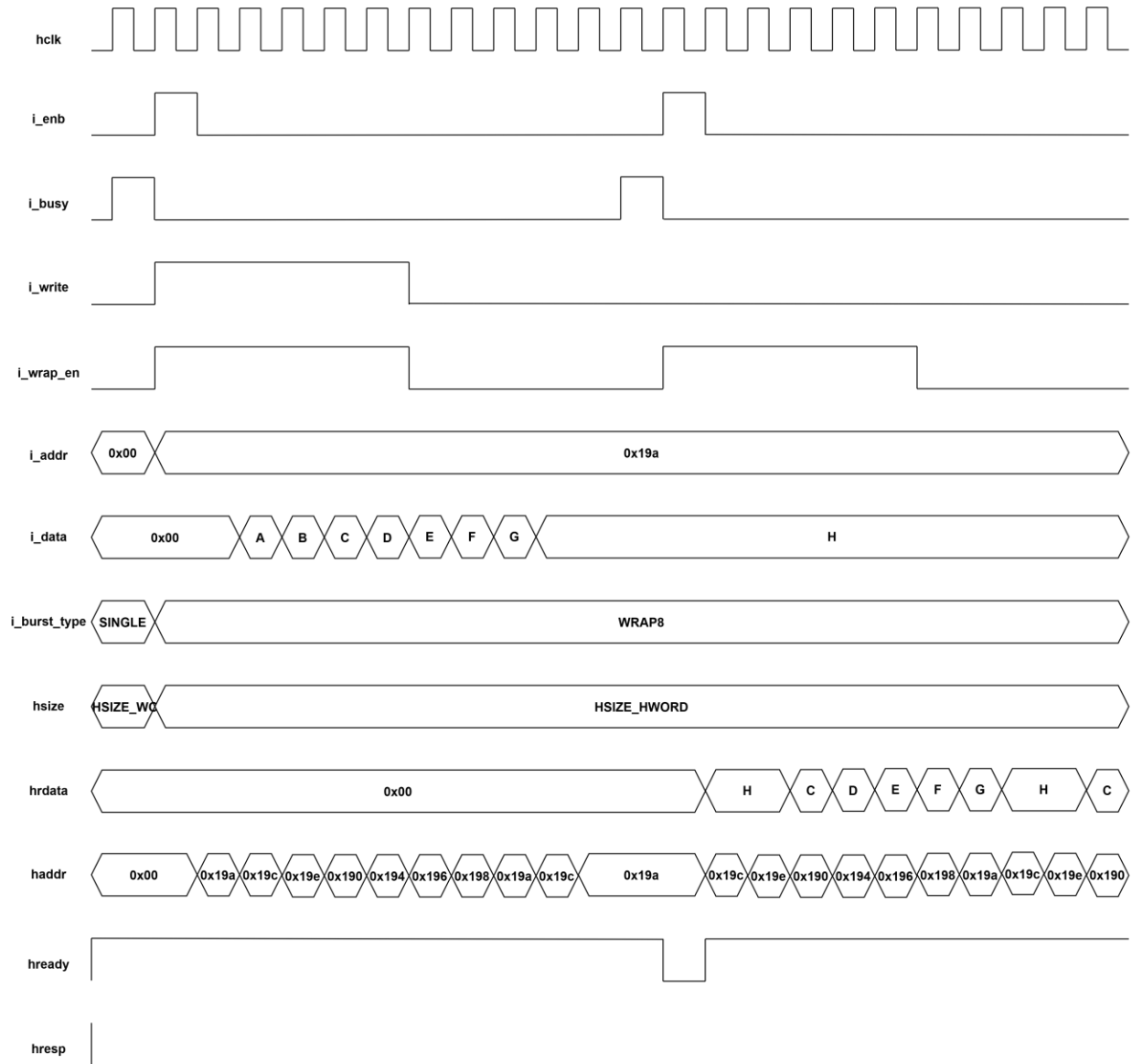


Figure 3.11. Waveform of WRAP8 burst transfer

Idle Phase

- The interface starts in the idle state with $i_enb = 0$ and the initial address set to 0x00.
- $hready$ remains high and $hresp$ remains low.

Address Phase – Burst Start

- i_addr changes from 0x00 to 0x19A, indicating the start address of the burst transfer.

- `i_burst_type = WRAP8`, indicating an 8-beat wrapping burst.
- `hsize = HSIZE_HWORD`, so each transfer occupies 2 bytes.
- The starting address `0x19A` is driven onto `haddr`.

Data Phase

- The transferred data progresses through A, B, C, D, E, F, G, and H.
- During the read transfer, `hrdata` returns H, C, D, E, F, G, H, and C as shown in the waveform.
- When `i_busy` is asserted, `hready` is driven low, inserting a wait state into the burst transfer.
- While `hready` is low, the current transfer is extended and the address remains unchanged until `hready` returns high.

Address Progression

- Since `HSIZE_HWORD` is selected, the address increases by 2 bytes for each beat.
- The address sequence is: `0x19A → 0x19C → 0x19E → 0x190 → 0x192 → 0x194 → 0x196 → 0x198`.
- The burst wraps within the 16-byte boundary `0x190–0x19F`.
- After reaching `0x19E`, the next address wraps to `0x190` instead of continuing to `0x1A0`.
- When `hready` is deasserted due to `i_busy`, address progression is temporarily stalled until `hready` returns high.
- The waveform shows the wrapped sequence continuing within the same boundary until all eight beats are completed.
- A subsequent sequence starts again from `0x19A` as shown in the waveform.

Burst Behavior

- `WRAP8` defines an eight-beat wrapping burst.
- The waveform shows consecutive halfword transfers with wrapping addresses.
- The address sequence follows the wrap pattern: `0x19A → 0x19C → 0x19E → 0x190 → 0x192 → 0x194 → 0x196 → 0x198`.
- The wrap boundary size is 16 bytes (8 beats × 2 bytes per beat).
- A wait state is inserted when `i_busy` is asserted and `hready` goes low.

- The burst resumes once hready returns high and continues from the stalled address.
- hresp = 0 throughout the waveform, indicating no error response is shown.

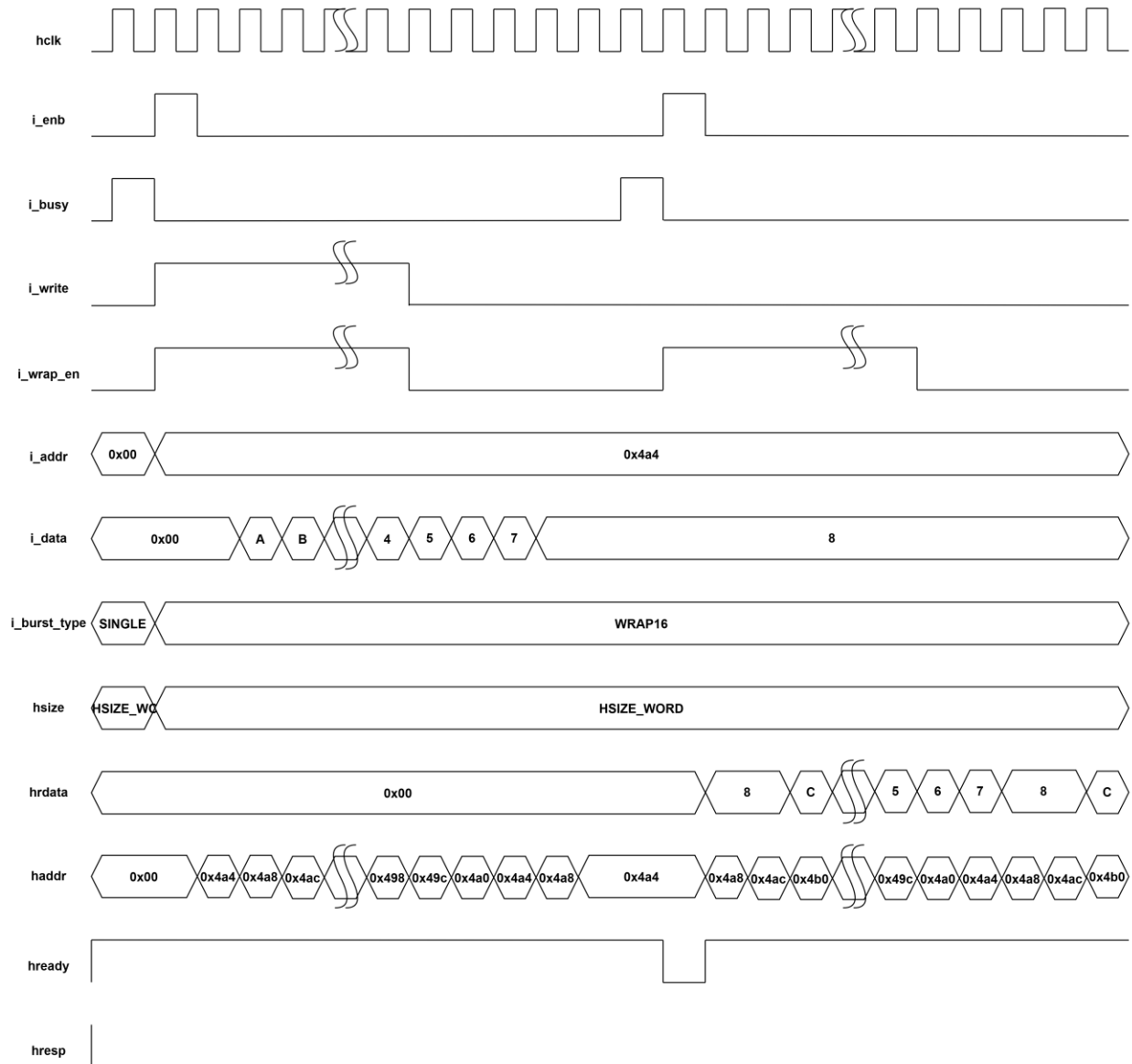


Figure 3.12. Waveform of WRAP16 burst transfer

Idle Phase

- The interface starts in the idle state with i_enb = 0 and the initial address set to 0x00.
- hready remains high and hresp remains low.

Address Phase – Burst Start

- `i_addr` changes from 0x00 to 0x4A4, indicating the start address of the burst transfer.
- `i_burst_type` = WRAP16, indicating a 16-beat wrapping burst.
- `hsize` = HSIZE_WORD, so each transfer occupies 4 bytes.
- The starting address 0x4A4 is driven onto `haddr`.

Data Phase

- The transferred data progresses through A, B, C, D, E, F, G, H, 1, 2, 3, 4, 5, 6, 7, and 8.
- During the read transfer, `hrdata` returns data corresponding to each wrapped address location.
- When `i_busy` is asserted, `hready` is driven low, inserting a wait state into the burst transfer.
- While `hready` is low, the current transfer is extended and the address remains unchanged until `hready` returns high.

Address Progression

- Since HSIZE_WORD is selected, the address increases by 4 bytes for each beat.
- The wrap boundary size is 64 bytes (16 beats × 4 bytes per beat).
- The address 0x4A4 belongs to the wrapping region 0x480–0x4BF.
- The address sequence is: 0x4A4 → 0x4A8 → 0x4AC → 0x4B0 → 0x4B4 → 0x4B8 → 0x4BC → 0x480 → 0x484 → 0x488 → 0x48C → 0x490 → 0x494 → 0x498 → 0x49C → 0x4A0.
- After reaching 0x4BC, the next address would be 0x4C0, which exceeds the upper wrap boundary (0x4BF), so the address wraps to 0x480.
- When `hready` is deasserted due to `i_busy`, address progression is temporarily stalled until `hready` returns high.
- The waveform shows the wrapped sequence continuing within the same 64-byte boundary until all sixteen beats are completed.
- A subsequent sequence starts again from 0x4A4 as shown in the waveform.

Burst Behavior

- WRAP16 defines a sixteen-beat wrapping burst.
- The waveform shows consecutive word transfers with wrapping addresses.
- The address sequence follows the wrap pattern: 0x4A4 → 0x4A8 → 0x4AC → 0x4B0 → 0x4B4 → 0x4B8 → 0x4BC → 0x480 → 0x484 → 0x488 → 0x48C → 0x490 → 0x494 → 0x498 → 0x49C → 0x4A0.
- The addresses wrap within the 64-byte aligned region 0x480–0x4BF.
- A wait state is inserted when i_busy is asserted and hready goes low.
- The burst resumes once hready returns high and continues from the stalled address.
- hresp = 0 throughout the waveform, indicating no error response is shown.